

**Национальный исследовательский университет «Высшая школа экономики»
Факультет информатики, математики и компьютерных наук
Образовательная программа «Прикладная математика и информатика»**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**«Оптимизация качества облачного видеостриминга в условиях ограниченности
вычислительных ресурсов с применением классических эвристических методов и
подходов на основе машинного обучения»**

Выполнил: студент 4 курса
Паршин Даниил Денисович

Научный руководитель:
Колданов П. А.

Соруководитель:
Жизлина В. Г.

Нижний Новгород, 2026

Аннотация

В выпускной квалификационной работе рассматривается задача повышения воспринимаемого качества облачного видеостриминга при ограниченных вычислительных ресурсах. Предлагаемый подход основан на выделении областей интереса в кадре и передаче их кодировщику как приоритетных зон. Центральным инженерным результатом является библиотека IDet — CPU-first C++ слой для формирования ROI, объединяющий три режима анализа: текстовые области, лица и текстурированные области одежды и материалов. Для текстовых областей рассматривается семейство DBNet/DBNet++, для лиц — семейство SCRFD, для текстурированных областей подготовлен отдельный детектор на базе YOLO и воспроизводимый конвейер подготовки данных, обучения и описания контракта экспорта модели.

Работа фокусируется не на изолированной точности одной нейросетевой модели, а на архитектуре библиотеки, жизненном цикле обработки кадра, унификации результатов разных детекторов, управлении буферами, политике потоков и преобразовании ROI-геометрии в управляющие подсказки для кодировщика. В текущем публичном API IDet результат детекции представлен как VecQuad; расширенная запись с типом ROI, приоритетом и `qoffset` рассматривается в работе как downstream-контракт интеграции с кодировщиком, а не как уже существующий публичный класс библиотеки. Дополнительно описан generic YOLO pipeline, включая согласование таксономии DeepFashion2 и Fashionpedia, аудит датасета, обучение детектора текстурированных областей и контракт экспорта PT-модели в ONNX-артефакт для последующего CPU-инференса через ONNX Runtime.

Ключевые слова: облачный видеостриминг; ROI-кодирование; видеокодирование; IDet; CPU inference; ONNX Runtime; DBNet; SCRFD; YOLO; DeepFashion2; Fashionpedia; textured ROI; VMAF; SSIM.

Содержание

Аннотация	2
Введение	6
1 Предметная область и постановка задачи	8
1.1 Облачный видеостриминг как задача распределения ресурсов	8
1.2 Почему глобальные метрики недостаточны	8
1.3 ROI-aware кодирование в существующих исследованиях	9
1.4 Три режима ROI в предлагаемой системе	9
2 Формальная постановка задачи ROI-aware оптимизации	11
2.1 Ограниченная оптимизация качества	11
2.2 Бюджет площади ROI	11
2.3 Временная стабильность ROI	12
2.4 От ROI к qoffset кодировщика	12
2.5 Метрики детекции	12
3 Проектирование IDet и трехрежимного ROI-конвейера	14
3.1 IDet как инженерное ядро работы	14
3.2 Жизненный цикл обработки кадра	16
3.3 Ответственность адаптеров детекторов	17
3.4 Представление ROI	17
3.5 Владение буферами и горячий путь	18
3.6 Приоритеты и разрешение конфликтов	19
3.7 Реальный публичный контракт IDet	20
3.8 Downstream-контракт ROI и приоритеты областей	21
3.9 Инварианты и отказоустойчивость IDet	22
3.10 Память, буферы и zero-allocation hot path	22
3.11 Политика потоков	22
3.12 Интеграция с кодировщиком	23
4 Детектор текстурированных ROI: данные, обучение и экспорт	25
4.1 Мотивация детекции текстурированных ROI	25
4.2 Конвейер подготовки и обучения YOLO-модели	25
4.3 Контракт PT-to-ONNX артефакта	26
4.4 Единая 7-классовая таксономия	27

4.5	Аудит датасетов после конвертации	28
4.6	Дисбаланс классов	29
4.7	Геометрия изображений и bbox	30
4.8	Режимы обучения	31
4.9	Артефакты ONNX Runtime и квантизация	32
5	Экспериментальные результаты YOLO-моделей на объединенной таксономии	36
5.1	Граница экспериментальных выводов	36
5.2	Аппаратная конфигурация	36
5.3	Итоговые метрики	36
5.4	Поклассовая точность детектора	37
5.5	Строгая поклассовая AP и сравнение моделей	38
5.6	Матрица ошибок	39
5.7	Почему выбран YOLO26n	41
6	Интеграция ROI в видеокодирование	43
6.1	ROI как управляющий сигнал, а не конечный результат	43
6.2	Политика qoffset	43
6.3	Fallback-режимы	43
6.4	Целевой эксперимент качества	44
7	Методика сравнения baseline и ROI-aware закодированного потока	45
7.1	Цель эксперимента	45
7.2	Экспериментальные ветки	45
7.3	Контроль переменных	45
7.4	ROI-взвешенные метрики качества	46
7.5	Форма фиксации итоговых метрик	46
7.6	Критерии подтверждения гипотезы	48
8	Экспериментальная методика и критерии качества	49
8.1	Метрики качества видео	49
8.2	Системные метрики	50
8.3	Метрики покрытия ROI	50
8.4	Дисциплина CPU-only benchmark	51
8.5	NUMA, Kunpeng и ONNX Runtime CPU Execution Provider	51
8.6	Кривые качества и вычислительной стоимости	52
8.7	Абляционное исследование	53
9	Экспериментальное исследование производительности IDet	54
9.1	Состав benchmark-эксперимента	54
9.2	Latency-quality trade-off	56
9.3	Интерпретация по режимам	57
9.4	Profiling и I/O Binding	57

9.5	Top-Down Microarchitecture Analysis	58
9.6	Ограничения текущих benchmark-данных	60
10	Инженерные ограничения, риски и практическая значимость	61
10.1	Задержка как главный ограничитель	61
10.2	Дисбаланс классов	61
10.3	Центральное смещение объектов	61
10.4	Риск ложных textured ROI	62
10.5	Практическая значимость	62
11	Производственная интеграция и план развития IDet	63
11.1	Почему детектор текстурированных ROI должен быть частью IDet	63
11.2	План интеграции YOLO26n для текстурированных ROI	63
11.3	Политика потоков при производственной интеграции	64
11.4	Профиль памяти и отсутствие динамических аллокаций в критическом пути .	64
11.5	План CPU-бенчмарка	65
11.6	Связь с качеством видеокодирования	65
11.7	Режимы эксплуатации	66
11.8	Научный аспект: текстурированные ROI как приближение к perceptual saliency	66
11.9	Инженерный аспект: преимущество объединенной таксономии	66
11.10	Ограничения валидности и требования к финальным экспериментам	67
	Заключение	68
	Список литературы	69
A	Дополнительные таблицы экспериментов	72
A.1	Сравнение датасетов	72
A.2	Матрицы ошибок YOLO-экспериментов	72
A.3	Параметризованный запуск C++ benchmark	73
A.4	Целевая процедура benchmark	73

Введение

Актуальность темы. Облачный видеостриминг является одним из наиболее ресурсоемких классов мультимедийных систем. В реальном сервисе качество видео ограничивается не только скоростью сети пользователя, но и вычислительной стоимостью транскодирования, задержкой доставки, пропускной способностью инфраструктуры, типом кодировщика и сложностью видеоконтента. Классические алгоритмы адаптивного выбора битрейта решают часть проблемы на уровне сегментов видео, подбирая представление потока во времени [1, 2]. Однако они практически не отвечают на вопрос о том, как распределять качество внутри одного кадра.

В пользовательском восприятии кадр не является равномерным. Для лекций, стримов, блогерского контента, обзоров товаров и видеоконференций особенно важны лица, текстовые элементы, субтитры, интерфейсные надписи и детали одежды или материала. Если кодировщик чрезмерно сглаживает лицо, делает текст нечитаемым или уничтожает фактуру одежды, субъективное качество снижается даже при приемлемом среднем битрейте. Поэтому в данной работе видео рассматривается не как однородная сетка пикселей, а как набор областей с различной зрительной ценностью.

Идея работы состоит в построении ROI-aware конвейера: сначала на кадре выделяются значимые области, затем они стабилизируются во времени, после чего передаются кодировщику как области с более высоким приоритетом. Такой подход не отменяет классическое видеокодирование и не заменяет ABR-логику, а дополняет их на пространственном уровне. При этом задача становится инженерно сложной: ROI-детекция должна быть достаточно быстрой, чтобы не уничтожить выигрыш от более рационального распределения битов.

Практическая основа работы — библиотека IDet [39], спроектированная как CPU-first C++ слой детекции и унификации ROI для real-time pipeline. В архитектуре библиотеки текстовые области могут обрабатываться адаптерами семейства DBNet/DBNet++ [16, 17], лица — адаптерами семейства SCRFD [18], а текстурированные области — отдельным YOLO-детектором. Дополнительным инженерным артефактом является generic YOLO training pipeline [41]: он отвечает за подготовку датасетов, перевод в YOLO-формат, аудит качества разметки, объединение таксономий, обучение, построение отчетов и экспорт обученной RT-модели в deployable ONNX-артефакт [26, 27].

Цель работы — разработать и обосновать инженерно-научный подход к повышению воспринимаемого качества облачного видеостриминга за счет выделения и защиты ROI в условиях ограниченного CPU-бюджета.

Для достижения цели поставлены следующие задачи:

- 1) проанализировать задачу облачного видеостриминга и ограничения, возникающие при транскодировании видео;
- 2) формализовать задачу ROI-aware оптимизации качества как задачу распределения вычислительных и битовых ресурсов;

- 3) разработать трехрежимную архитектуру ROI-детекции: текст, лицо, текстурированные области;
- 4) описать IDet как практическую CPU-oriented библиотеку для детекции и формирования ROI;
- 5) подготовить YOLO-детектор текстурированных ROI на общей таксономии DeepFashion2 и Fashionpedia;
- 6) сравнить отдельные и объединенные датасеты, выявить дисбаланс классов и свойства геометрии bbox;
- 7) обосновать выбор YOLO26n как компромисса между точностью и задержкой;
- 8) предложить схему интеграции ROI с кодировщиком через qoffset/ROI metadata и fallback-режимы;
- 9) сформировать протокол экспериментальной оценки качества, скорости и устойчивости системы.

Объект исследования — облачный конвейер видеостриминга и транскодирования.

Предмет исследования — методы выделения и использования областей интереса для повышения воспринимаемого качества видео при ограниченных вычислительных ресурсах.

Научно-практическая новизна работы заключается в том, что ROI-детекция рассматривается не как самостоятельная задача компьютерного зрения, а как элемент системной оптимизации видеокодирования. В отличие от сценария, где модель оценивается только по mAP, здесь качество модели анализируется вместе с latency, memory footprint, предсказуемостью pre/post-processing и способностью работать внутри CPU-first C++ библиотеки.

1. Предметная область и постановка задачи

1.1 Облачный видеостриминг как задача распределения ресурсов

Облачный видеостриминг строится вокруг компромисса между качеством, стоимостью и задержкой. В простейшей постановке платформа получает исходный видеопоток, транскодирует его в несколько представлений и доставляет пользователю версию, соответствующую текущим сетевым условиям. На практике этот процесс дополняется ограничениями инфраструктуры: количество параллельных потоков, CPU/GPU budget, стоимость хранения, требования к latency и устойчивость качества на разных устройствах.

Классические ABR-алгоритмы, такие как BOLA [1], и learning-based подходы, такие как Pensieve [2], оптимизируют выбор битрейта во времени. Их цель — выбрать сегмент нужного качества, не допустить rebuffering и сохранить стабильность воспроизведения. Но даже идеальный выбор битрейта не гарантирует, что кодировщик правильно распределит качество внутри кадра. На одном и том же битрейте можно сохранить фон и потерять лицо, либо наоборот защитить лицо и текст, пожертвовав малозначимыми областями.

Видеокодеки H.264/AVC и HEVC/H.265 основаны на блочной структуре, предсказании, преобразовании, квантовании и энтропийному кодированию [6, 7, 8, 9]. Потери качества в основном связаны с квантованием, поэтому управление параметром квантования становится естественной точкой воздействия. Если некоторая область кадра важнее остальных, то для нее можно использовать меньший QP или отрицательный qoffset. Именно такая идея лежит в основе ROI-aware кодирования.

1.2 Почему глобальные метрики недостаточны

Общие метрики качества, такие как PSNR, SSIM [3], VMAF [4] и LPIPS [5], полезны для сравнения кодеков и режимов кодирования, но они не всегда отражают пользовательское восприятие локальных дефектов. Если кодировщик ухудшил небольшую область с лицом или текстом, глобальная метрика может измениться мало, но пользователь заметит деградацию сразу.

Поэтому для данной работы важен переход от глобального качества к ROI-weighted quality. Пусть кадр содержит множество областей R_i с весами w_i , отражающими их perceptual importance. Тогда агрегированное качество можно записать как

$$Q_{\text{ROI}}(F) = \frac{\sum_{i=1}^N w_i \cdot Q(F|_{R_i})}{\sum_{i=1}^N w_i}, \quad (1.1)$$

где $F|_{R_i}$ означает фрагмент кадра внутри области R_i , а $Q(\cdot)$ — локальная метрика качества. Такая формула не заменяет VMAF или SSIM, а задает способ интерпретировать их в привязке к областям интереса.

1.3 ROI-aware кодирование в существующих исследованиях

Идея защиты пространственно важных областей не является специфичной только для данной работы. В исследованиях по ROI-based video coding рассматриваются схемы, в которых битовый бюджет перераспределяется между областями кадра с разной семантической или зрительной значимостью [13, 14, 15]. В таких работах обычно выделяются два независимых вопроса: как определить ROI и как затем встроить эту информацию в rate-control или quantization decision кодировщика. В данной ВКР эти вопросы также разделены: IDet отвечает за построение списка областей, а encoder adapter преобразует их в qoffset/ROI metadata или fallback-политику.

Системная сложность ROI-aware подхода состоит в том, что улучшение локального качества не должно достигаться любой ценой. Если ROI занимает слишком большую часть кадра, кодировщик теряет возможность перераспределить биты; если ROI нестабилен во времени, качество может мерцать; если детектор слишком медленный, рост качества будет куплен нарушением latency budget. Поэтому в данной работе ROI-aware кодирование рассматривается как ограниченная оптимизационная задача, а не как простое добавление ещё одной нейросетевой модели.

1.4 Три режима ROI в предлагаемой системе

В итоговой архитектуре IDet рассматривается как библиотека, поддерживающая три режима ROI:

- 1) **text ROI** — области текста, субтитров, интерфейса и мелких надписей;
- 2) **face ROI** — лица ведущих, участников и людей, доминирующих в кадре;
- 3) **textured ROI** — области одежды, ткани, ворса, декоративных поверхностей и других структур, потеря которых ухудшает воспринимаемую детализацию.

Текстовая ветка опирается на идею сегментационной детекции текста. DBNet встраивает дифференцируемую бинаризацию в модель, что уменьшает зависимость от внешней постобработки и делает метод удобным для потокового сценария [16]. DBNet++ развивает этот подход за счет adaptive scale fusion, что полезно для текста разных размеров и пропорций [17]. Ветвь лиц основана на SCRFD: это семейство детекторов проектировалось как эффективный компромисс точности и вычислительной стоимости за счет перераспределения обучающих примеров и вычислений между частями модели [18]. Третья ветка добавляет textured ROI: она не заменяет text/face detection, а закрывает область, для которой готовых универсальных решений в контексте ROI-кодирования недостаточно.

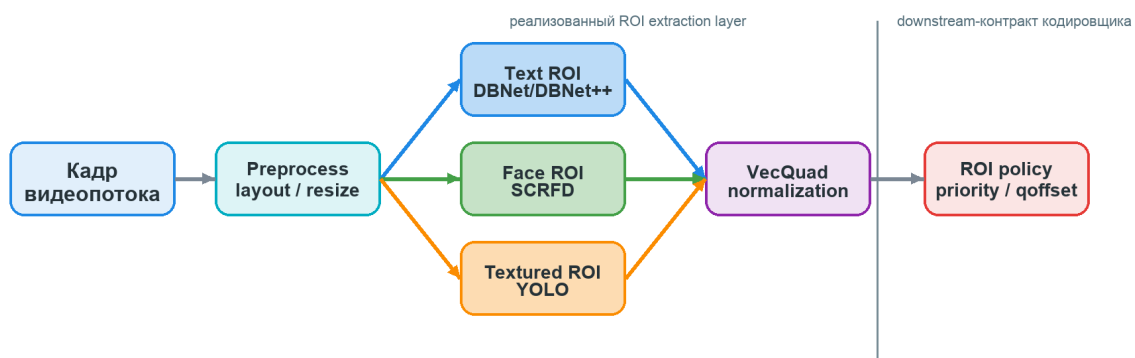


Рисунок 1.1: Архитектура ROI-aware конвейера на базе IDet: библиотека объединяет text ROI, face ROI и textured ROI в геометрический выход, а downstream-слой преобразует его в приоритеты для кодировщика.

2. Формальная постановка задачи ROI-aware оптимизации

2.1 Ограниченная оптимизация качества

В работе задача улучшения качества видео формулируется как ограниченная оптимизация. Пусть B — доступный битрейт, T — допустимая задержка анализа кадра, C — вычислительный бюджет, θ — параметры ROI-детектора и политики кодирования. Тогда целевую постановку можно записать так:

$$\max_{\theta} \mathbb{E}[Q_{\text{ROI}}(F; \theta)] \quad \text{при условиях} \quad R(F; \theta) \leq B, \quad L(F; \theta) \leq T, \quad C(F; \theta) \leq C_{\text{max}}. \quad (2.1)$$

Здесь R — фактический расход битрейта, L — задержка обработки кадра, C — стоимость вычислений, а Q_{ROI} — ROI-взвешенное качество. В отличие от лабораторной постановки, где можно максимизировать только mAP, в данной работе важна системная метрика: улучшение качества должно быть достигнуто без выхода за latency budget.

2.2 Бюджет площади ROI

Если защищать слишком большую часть кадра, кодировщик потеряет возможность перераспределять биты. Поэтому вводится ограничение на долю защищенной площади:

$$\frac{\left| \bigcup_{i=1}^N R_i \right|}{|F|} \leq \alpha, \quad (2.2)$$

где α — максимально допустимая доля кадра, например 0.15–0.35 в зависимости от типа контента. Это ограничение принципиально важно для практического стриминга: ROI-aware кодирование не должно превращаться в кодирование всего кадра с повышенным качеством.

Бюджет площади нужен не только как математическое ограничение, но и как инженерный предохранитель от ложных срабатываний. Если text ROI обычно занимает небольшую долю кадра и имеет высокий приоритет, то textured ROI потенциально может покрыть крупные области одежды, мебели или фона. Без ограничения площади такой режим начнет конкурировать с лицами и текстом за битовый бюджет, хотя его perceptual importance ниже. Поэтому площадь должна учитываться после слияния областей, а не только на уровне каждого отдельного детектора: несколько корректных ROI по отдельности могут вместе занять слишком большую часть кадра.

Практическая политика может задавать разные значения α для разных типов контента. Для лекции с субтитрами допустима небольшая площадь text ROI и высокий qoffset; для обзора товара или fashion-сцены допустима более крупная textured ROI; для видеоконференции приоритетнее лицо, а текстурированные области должны быть ограничены сильнее. Таким образом, α является параметром runtime policy, а не универсальной константой модели.

2.3 Временная стабильность ROI

Для видео важна не только точность областей в отдельном кадре, но и их стабильность во времени. Если ROI резко прыгает между соседними кадрами, это может привести к мерцанию качества. Для сглаживания confidence используется рекуррентная формула

$$\hat{s}_t = \lambda \hat{s}_{t-1} + (1 - \lambda) s_t, \quad (2.3)$$

где s_t — текущая уверенность детектора, $\hat{s}_t$ — сглаженная уверенность, $\lambda \in [0, 1]$ — коэффициент памяти. Дополнительно можно ввести штраф за нестабильность:

$$P_{flicker} = \frac{1}{T-1} \sum_{t=2}^T (1 - \text{IoU}(R_t, R_{t-1})). \quad (2.4)$$

Чем выше $P_{flicker}$, тем сильнее меняется ROI между кадрами и тем выше риск визуального мерцания.

2.4 От ROI к qoffset кодировщика

FFmpeg предоставляет структуру `AVRegionOfInterest`, где область задается координатами и параметром `qoffset`; отрицательное значение `qoffset` означает повышение качества кодирования внутри ROI [10]. На уровне фильтров аналогичная идея доступна через `addroi`: фильтр не изменяет пиксели кадра, а прикрепляет к нему `metadata` об областях интереса для последующего кодирования [11]. Упрощенная формула преобразования приоритета p_i в смещение QP может быть записана как

$$\Delta QP_i = -\gamma \cdot p_i \cdot (QP_{\max} - QP_{\min}), \quad (2.5)$$

где $p_i \in [0, 1]$ — нормированный приоритет области, а γ — коэффициент силы защиты. Формула (2.5) не навязывает конкретную реализацию кодировщика, но описывает общий принцип: чем важнее область, тем сильнее отрицательное смещение QP.

2.5 Метрики детекции

Для модели `textured ROI` используются стандартные метрики `object detection`: `precision`, `recall`, `F1`, `AP`, `mAP@0.5` и `mAP@0.5:0.95` [28]. Они вычисляются по формулам

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (2.6)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2.7)$$

В контексте ROI-задачи `recall` особенно важен: пропуск значимой области одежды хуже, чем неточная классификация типа одежды, потому что цель детектора текстурированных ROI — не модная аналитика, а защита визуально богатой области от агрессивного сжатия.

AP измеряет площадь под `precision–recall` кривой при заданном IoU-пороге, а

$mAP@0.5:0.95$ усредняет AP по нескольким порогам IoU от 0.5 до 0.95. Поэтому $mAP@0.5$ показывает, насколько модель в целом находит объекты, а $mAP@0.5:0.95$ строже оценивает геометрию bbox. Для ROI-aware кодирования это различие принципиально: слишком грубый bbox может защищать лишний фон, а слишком маленький bbox может не покрыть значимую фактуру. Поэтому в работе отдельно анализируются как стандартные detection metrics, так и area-based метрики покрытия ROI.

Матрица ошибок дополняет mAP и показывает, какие классы путаются между собой. В задаче textured ROI такая путаница не всегда критична: если модель перепутала юбку и платье, но корректно выделила текстурированную область, кодировщик все равно получает полезный сигнал. Более опасны ошибки другого типа: пропуск одежды как background или выделение background как важной textured ROI. Поэтому интерпретация confusion matrix в данной работе отличается от классической задачи fine-grained fashion classification.

3. Проектирование IDet и трехрежимного ROI-конвейера

3.1 IDet как инженерное ядро работы

IDet — это CPU-first C++ библиотека для детекции и нормализации областей интереса [39]. В данной работе она рассматривается как центральный инженерный результат: задача библиотеки состоит не в том, чтобы обучить единственную модель, а в том, чтобы связать несколько специализированных детекторов с видеокодировщиком через устойчивый геометрический контракт. Реальный публичный контракт IDet возвращает список четырехугольников `VecQuad`; типизация ROI, приоритеты и `goffset` являются следующим слоем интеграции, который строится поверх этой геометрии при подключении кодировщика. Такой дизайн отделяет компьютерное зрение от политики кодирования и делает систему расширяемой: можно заменить модель текста, лиц или текстурированных областей, не меняя downstream-логику распределения качества.

Архитектурно IDet опирается на несколько принципов. Во-первых, критический путь должен работать на CPU без Python-зависимости, поскольку облачный transcoding часто выполняется на CPU-пулах или в средах, где GPU является дорогим и разделяемым ресурсом. Во-вторых, модели должны приводиться к deployable формату ONNX, а инференс должен выполняться через ONNX Runtime [19, 20]. В-третьих, графы моделей должны проходить оптимизацию: ONNX Runtime поддерживает уровни graph optimizations и online/offline режимы, что позволяет отделить подготовку артефакта от исполнения в сервисе [22]. В-четвертых, входные и выходные буферы должны быть переиспользуемыми; механизм I/O Binding в ONNX Runtime позволяет заранее связать входы и выходы с нужным размещением и уменьшить количество копирований [21].

Таблица 3.1: Модули IDet и downstream-слои интеграции с кодировщиком

Модуль	Ответственность	Выходной результат
Frame input layer	Прием кадра, проверка формата, масштабирование и подготовка рабочего представления.	Нормализованный кадр или набор плиток.
Preprocessing layer	Letterbox, resize, нормализация каналов, преобразование layout и заполнение входных tensor buffers.	Готовые входные буферы для ONNX Runtime.
Model adapters	Обертки над text, face и texture detectors; скрывают формат выходов конкретной модели.	Сырые кандидаты: box, score, class/kind.
Post-processing layer	Thresholding, декодирование координат, NMS, stitching плиток и обратное преобразование координат.	Кандидаты ROI в координатах исходного кадра.
Downstream ROI integration	Добавление типа ROI, приоритета, area cap и temporal smoothing поверх геометрии VecQuad.	Упорядоченный список ROI metadata для кодировщика.
Encoder adapter	Преобразование ROI metadata в qoffset/ROI side data или fallback-политику.	Encoder guidance для видеокодировщика.
Benchmark layer	Измерение latency, числа ROI, площади ROI, числа аллокаций и стабильности результатов.	p50/p95/p99, отчеты и диагностические метрики.

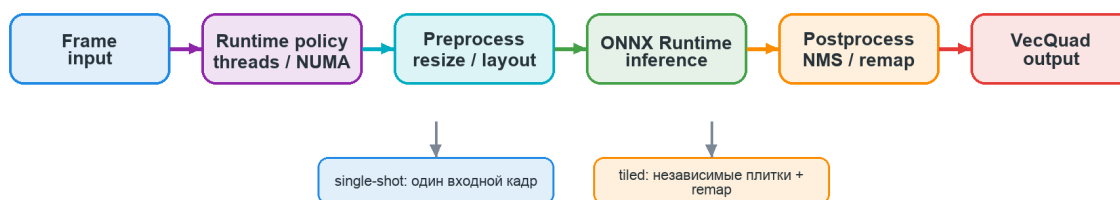


Рисунок 3.1: Runtime path IDet для одного кадра: подготовка изображения, CPU-инференс, tiled/single post-processing и возврат списка VecQuad.

3.2 Жизненный цикл обработки кадра

Жизненный цикл кадра в IDet начинается до запуска нейросетевой модели. Сначала система получает кадр в исходном разрешении, проверяет формат пикселей и выбирает режим обработки: полный кадр, downscale, tiled inference или повторное использование ROI с предыдущего кадра. Затем выполняется предобработка, включая letterbox/resize и приведение данных к формату, ожидаемому конкретной ONNX-моделью.

После инференса каждый движок декодирует выходы своей модели. Text engine преобразует probability map DBNet-like модели в quadrilateral detections, SCRFD engine декодирует multi-head face outputs, а YOLO engine преобразует raw или in-graph-NMS outputs в четырехугольники. Публичный слой возвращает VecQuad; следующий интеграционный слой должен добавить тип области, приоритет, ограничение суммарной площади и temporal smoothing. Последним шагом encoder adapter переводит такую ROI metadata в qoffset или другой backend-specific hint.

Таблица 3.2: Жизненный цикл кадра в ROI-aware конвейере на базе IDet

Шаг	Этап	Смысл этапа
1	Frame acquisition	Получение кадра и фиксация его размеров, timestamp и pixel format.
2	Work plan	Выбор режима обработки: полный кадр, плитки, пропуск тяжелого детектора или перенос ROI.
3	Preprocessing	Подготовка входных буферов: resize/letterbox, нормализация, layout conversion.
4	Inference	Выполнение одной или нескольких ONNX Runtime sessions с контролируемым числом потоков.
5	Model post-processing	Декодирование выходов моделей, thresholding, NMS, stitching и обратное масштабирование координат.
6	Downstream integration	ROI Добавление типа ROI, сортировка, слияние пересечений, area cap и temporal smoothing поверх VecQuad.
7	Encoder guidance	Преобразование ROI в qoffset/side data или fallback-параметры кодировщика.
8	Telemetry	Запись latency, числа ROI, площади ROI и диагностической информации.

3.3 Ответственность адаптеров детекторов

Adapter в IDet отвечает за преобразование конкретной модели в общий контракт библиотеки. Он не должен принимать решения о том, какие ROI важнее для кодировщика; это ответственность ROI manager. Такой принцип уменьшает связность: модель текста может быть заменена с DBNet на DBNet++, модель лиц — на другую SCRFD-конфигурацию, а детектор текстурированных ROI — на более тяжелую YOLO-модель без изменения downstream-слоя.

Таблица 3.3: Ответственность адаптеров IDet

Adapter	Внутренняя ответственность	Что не входит в ответственность
Text adapter	Предобработка под DBNet/DBNet++, декодирование text maps, polygon/box extraction, фильтрация по confidence.	Глобальный приоритет текста относительно лица и textured ROI.
Face adapter	Resize/normalize под SCRFD, декодирование anchors/candidates, NMS лиц, нормализация confidence.	Решение о qoffset и ограничение общей площади ROI.
Texture adapter	YOLO-preprocessing, декодирование box/class outputs, NMS, remap координат из letterbox-пространства.	Интерпретация классов одежды как политик кодирования.
Encoder adapter	Преобразование ROI metadata в FFmpeg ROI side data, qoffset-map или fallback-параметры.	Запуск детекторов и изменение outputs моделей.

3.4 Представление ROI

Фактическое представление результата в публичном API IDet минималистично: библиотека возвращает VecQuad, то есть список четырехугольников в координатах изображения. На этапе постобработки конкретные движки используют модельно-зависимые кандидаты с confidence score и, для YOLO, классом объекта, но эти поля не являются частью публичного результата `Detector::detect`. Для интеграции с кодировщиком одного геометрического контракта недостаточно, поэтому в работе отдельно описывается downstream-запись ROI: она должна добавлять к Quad тип области, приоритет и рекомендуемое смещение QR.

Таблица 3.4: Статус компонентов ROI-aware конвейера

Компонент	Статус	Роль в работе
VecQuad	реализовано в IDet	Текущий публичный геометрический контракт: список четырехугольников в координатах изображения.
Text ROI detector	реализовано / интегрировано	Детекция текстовых областей, критичных для читаемости.
Face ROI detector	реализовано / интегрировано	Детекция лиц как семантически важных областей кадра.
Textured ROI detector	подготовлен через YOLO pipeline	Детекция одежды и фактурированных областей для третьего ROI-режима.
RichROI / ROI metadata	downstream-контракт	Расширение геометрии типом области, confidence, приоритетом и подсказкой qoffset.
qoffset mapping	проектируемая encoder policy	Преобразование приоритета ROI в управляющий сигнал для кодировщика.
Baseline vs ROI-aware quality	экспериментальный протокол	Финальная количественная оценка закодированного потока выполняется отдельной серией экспериментов.

Такое разделение предотвращает смешение уровней реализации. В текущем коде IDet уже существует единый геометрический выход и runtime-инфраструктура для CPU-инференса. Расширенная ROI metadata, приоритеты и qoffset относятся к downstream-интеграции с кодировщиком: они необходимы для полной ROI-aware системы, но не должны описываться как уже существующий публичный API библиотеки.

3.5 Владение буферами и горячий путь

Для real-time видеостриминга важна не только средняя задержка, но и устойчивость p95/p99. Динамические аллокации в критическом пути приводят к джиттеру: отдельные кадры внезапно обрабатываются дольше из-за выделения памяти, освобождения памяти или перераспределения контейнеров. Поэтому IDet должен использовать явную модель владения буферами.

Таблица 3.5: Модель владения буферами в IDet

Буфер	Владелец	Правило использования
Decoded frame	Frame input layer	Только чтение после получения кадра; исходное изображение не меняется детекторами.
Preprocessed input	Preprocessing layer	Переиспользуется между кадрами одного размера; заполняется перед inference.
ONNX input/output tensors	Inference backend	Выделяются при инициализации session или при смене input shape.
Candidate arrays	Model adapter	Очищаются и переиспользуются; вместимость растет только вне hot path.
NMS workspace	Post-processing layer	Общий временный буфер для сортировки и подавления кандидатов.
ROI metadata list	ROI manager / encoder integration layer	Финальный список областей с типом и приоритетом, передаваемый encoder adapter и telemetry layer.

3.6 Приоритеты и разрешение конфликтов

В системе вводится иерархия приоритетов: текст > лицо > textured ROI > фон. Текст получает максимальный приоритет, потому что даже небольшие артефакты делают его нечитаемым. Лицо получает высокий приоритет из-за чувствительности пользователя к искажениям человеческой внешности. Текстурированные поверхности получают средний приоритет: они важны для субъективной детализации, но не должны вытеснять лицо и текст.

Таблица 3.6: Алгоритм слияния ROI

Шаг	Действие
1	Получить кандидаты ROI от text adapter, face adapter и texture adapter.
2	Нормировать координаты, confidence и тип области к downstream-записи ROI поверх VecQuad.
3	Удалить кандидаты с confidence ниже порога, зависящего от типа ROI.
4	Отсортировать ROI по приоритету: text > face > texture > background.
5	Объединить сильно пересекающиеся области с учетом IoU и типа области.
6	Ограничить суммарную защищаемую площадь кадра заданным area budget.
7	Применить temporal smoothing к координатам и confidence для уменьшения flickering.
8	Вернуть упорядоченный список ROI metadata для encoder adapter.

3.7 Реальный публичный контракт IDet

Важное архитектурное решение IDet состоит в том, что конкретная модель не протекает в пользовательский код. Публичный слой выбирает задачу и движок через Task и EngineKind, а пользователь получает геометрию областей как VecQuad. Ниже приведен сокращенный фрагмент публичного заголовочного интерфейса библиотеки; он важен для диплома, потому что фиксирует фактическую границу между C++ библиотекой и внешним видеоконвейером.

```
1 enum class Task : std::uint8_t {
2     None = 0,
3     Text = 1,
4     Face = 2,
5     Cloth = 3,
6 };
7
8 enum class EngineKind : std::uint8_t {
9     None = 0,
10    DBNet = 1,
11    SCRFD = 2,
12    Yolo = 3,
13 };
14
15 using Quad = std::array<Point2f, 4>;
16 using VecQuad = std::vector<Quad>;
17
18 struct DetectorConfig final {
19     Task task = Task::None;
20     EngineKind engine = EngineKind::None;
21     InferenceOptions infer{};
22     RuntimePolicy runtime{};
23     std::string model_path{};
24     [[nodiscard]] Status validate() const noexcept;
25     static DetectorConfig setup(Task task, std::string model_path);
26 };
27
28 class Detector final {
29 public:
30     static Result<Detector> create(const DetectorConfig& config)
31         noexcept;
32     Status prepare_binding(int width, int height, int contexts)
33         noexcept;
34     Result<VecQuad> detect(const Image& image) noexcept;
35     Result<VecQuad> detect_bound(const Image& image, int ctx_idx)
```

```
noexcept;
};
```

Листинг 3.1: Сокращенный фрагмент публичного API IDet

Этот контракт задает границы ответственности фактической реализации. Модельно-зависимые детали — letterbox, layout conversion, нормализация входа, декодирование выходов, thresholding и NMS — скрыты внутри движков DBNet, SCRFD и YOLO. Downstream-слой, необходимый для ROI-aware кодирования, должен добавлять к возвращенной геометрии тип области, приоритет и qoffset; в текущем исходном коде это не отдельный публичный класс, а проектируемый слой интеграции с кодировщиком.

3.8 Downstream-контракт ROI и приоритеты областей

Унифицированное представление ROI для кодировщика должно содержать не только координаты. Для кодировщика важны тип области, уверенность, приоритет и ожидаемая сила защиты. Поэтому поверх фактического VecQuad в работе вводится downstream-запись ROI, которую можно реализовать в encoder integration layer. Минимальный состав такой записи приведен в таблице 3.7; это не утверждение о наличии одноименной структуры в текущем публичном API IDet.

Таблица 3.7: Downstream-контракт ROI metadata между детекторами, ROI manager и encoder adapter

Поле	Пример		Назначение
kind	text,	face,	Тип ROI, определяющий приоритет и политику qoffset.
	texture		
quad/bbox	TL–TR–BR–BL	или	Геометрия области в координатах исходного кадра.
	x1, y1, x2, y2		
confidence	0.0–1.0		Уверенность конкретного детектора.
priority	0.0–1.0		Системный приоритет после учета типа и confidence.
area	доля кадра		Используется для area budget и отсеечения чрезмерно больших ROI.
qoffsetHint	отрицательное значение		Предлагаемое смещение квантования для encoder adapter.
source	dbnet,	scrfd,	Источник ROI для диагностики и ablation study.
	yolo26n		
trackId	целое число или none		Связь ROI между кадрами для temporal smoothing.

Приоритеты должны быть заданы явно. В типичном сценарии текст получает наивысший приоритет, потому что потеря читаемости является критической. Лица также имеют высокий приоритет из-за заметности артефактов. Textured ROI полезны для perceived sharpness, но их приоритет должен быть ниже, чтобы они не вытесняли текст и лица при ограниченном

area budget.

3.9 Инварианты и отказоустойчивость IDet

Для производственного конвейера важно определить не только идеальный путь, но и инварианты, которые должны сохраняться при ошибках. IDet должен выдерживать отсутствие модели, пустой список ROI, недоступность native ROI support в кодировщике и временные пики задержки. Поэтому для каждого кадра система должна гарантировать следующие свойства:

- 1) координаты ROI находятся внутри границ исходного кадра;
- 2) суммарная площадь защищаемых областей не превышает заданный бюджет α ;
- 3) пересекающиеся ROI не приводят к конфликтующим qoffset одного и того же блока;
- 4) при сбое одного детектора остальные режимы могут продолжить работу;
- 5) при превышении latency budget система может временно перейти к переносимым ROI предыдущих кадров или к более дешевой ветке;
- 6) отсутствие native ROI support не должно ломать кодирование: encoder adapter обязан перейти к fallback-политике.

Эти инварианты важны для диплома, поскольку показывают, что IDet проектируется не как демонстрационный скрипт, а как слой, который может быть встроен в реальный видеоконвейер с ограничениями по времени, памяти и надежности.

3.10 Память, буферы и zero-allocation hot path

Одним из источников нестабильности p95/p99 является динамическое выделение памяти в горячем пути. Для CPU-first inference это особенно важно: даже если сама нейросеть выполняется быстро, повторяющиеся аллокации входных тензоров, временных массивов NMS или списков кандидатов могут ухудшать хвост задержек. Поэтому IDet должен стремиться к zero-allocation hot path: крупные буферы выделяются на этапе инициализации, а затем переиспользуются.

Практический список буферов включает входной image buffer, tensor input buffer, output buffers ONNX Runtime, временные массивы кандидатов, workspace NMS, массивы для stitching плиток и финальный контейнер VecQuad. Использование ONNX Runtime I/O Binding позволяет заранее связать входы и выходы с выбранными буферами и уменьшить количество лишних копирований между этапами [21]. Это особенно важно для ARM/Kunpeng CPU-only сценариев, где пропускная способность памяти и NUMA locality заметно влияют на tail latency.

3.11 Политика потоков

IDet должен контролировать два уровня параллелизма: внешний уровень обработки кадров/плиток и внутренний уровень потоков ONNX Runtime. Если оба уровня включены без

ограничений, возникает oversubscription: несколько пулов потоков конкурируют за одни и те же CPU-ядра, что ухудшает p95/p99 latency. Поэтому политика потоков должна задаваться явно и быть частью конфигурации, а не побочным эффектом backend.

Таблица 3.8: Политика потоков в IDet

Уровень	Настраиваемый параметр	Инженерный смысл
Frame-level parallelism	Число одновременно обрабатываемых кадров.	Увеличивает throughput, но может ухудшить latency отдельного кадра.
Tile-level parallelism	Число плиток, обрабатываемых параллельно.	Полезно для крупных кадров, но требует аккуратного stitching.
ORT intra-op threads	Число потоков внутри оператора ONNX Runtime.	Влияет на скорость сверточных и матричных операций.
ORT inter-op threads	Параллелизм между независимыми узлами графа.	Эффективен не для всех моделей; требует профилирования.
Detector sampling rate	Частота запуска каждой ветки ROI.	Позволяет запускать тяжелые детекторы реже и переносить ROI во времени.

3.12 Интеграция с кодировщиком

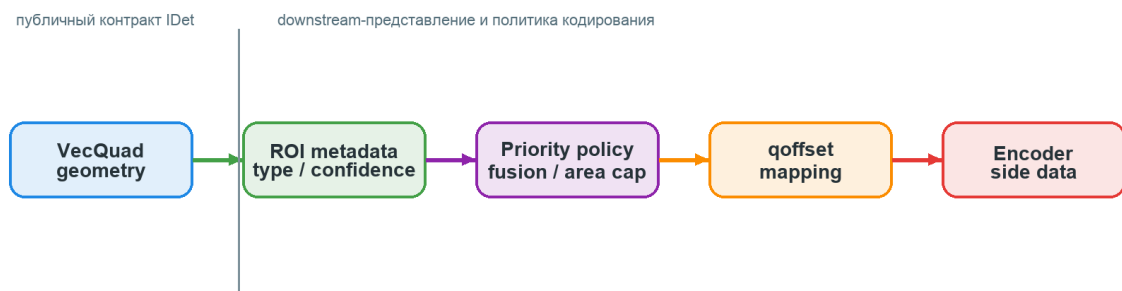


Рисунок 3.2: Преобразование ROI metadata в encoder guidance: ограничение площади, сортировка, qoffset mapping и передача ROI side data.

Интеграционный слой должен быть отделен от детекторов. Модель не должна знать, работает ли кодировщик через FFmpeg ROI metadata, adaptive quantization или через fallback-предобработку фона. FFmpeg addroi прикрепляет к кадру metadata об областях интереса, не изменяя пиксели самого кадра, а AVRegionOfInterest задает координаты и qoffset для таких областей [11, 10]. Это делает encoder adapter естественной границей между компьютерным зрением и кодированием.

Таблица 3.9: Алгоритмический контракт encoder adapter

Операция	Семантика
supportsROI	Проверить, может ли целевой backend принять ROI side data или qoffset-map.
mapPriority	Преобразовать приоритет ROI metadata в qoffset или эквивалентный encoder hint.
applyHints	Передать кодировщику список ROI после ограничения площади и сортировки.
fallback	При отсутствии native ROI support применить альтернативную стратегию: сглаживание фона или conservative AQ settings.

Этот adapter позволяет сохранить модульность: IDet отвечает за поиск областей, ROI manager — за слияние и стабилизацию, encoder adapter — за конкретный способ влияния на кодирование.

4. Детектор текстурированных ROI: данные, обучение и экспорт

4.1 Мотивация детекции текстурированных ROI

ROI-aware видеокодирование часто связывают с лицами и текстом, поскольку эти области имеют очевидную семантическую ценность. Однако в пользовательском видео существенную часть ощущения резкости формируют текстуры: одежда ведущего, ткань, ворс, узор, мелкие регулярные структуры и декоративные поверхности. При сильном сжатии такие области теряют высокочастотные детали и превращаются в сглаженные пятна. Поэтому textured ROI рассматривается как третий режим IDet: он не конкурирует с text ROI и face ROI, а расширяет систему на визуально богатые области, для которых трудно задать простые эвристики.

Для этой ветки используется объектный детектор семейства YOLO, обученный на согласованной таксономии одежды. Выбор объектного детектора, а не общей saliency-модели, связан с инженерной управляемостью: bbox легко преобразовать в ROI кодировщика, ошибки можно анализировать через AP и confusion matrix, а latency можно профилировать на CPU.

4.2 Конвейер подготовки и обучения YOLO-модели

Для обучения детектора текстурированных ROI используется отдельный проект generic YOLO training pipeline [41]. Его задача — обеспечить полный цикл подготовки и проверки модели: перевод исходных аннотаций в YOLO-представление, согласование таксономии, аудит разметки, управляемую подготовку датасета, запуск обучения, построение AP-отчетов, экспорт ONNX и последующую runtime-ориентированную оптимизацию. Это принципиально отличает pipeline от разового training script: модель не считается готовой после появления checkpoint, пока не проверены статистика датасета, поклассовые метрики, confusion matrix, переносимость ONNX-графа и поведение deploy-артефактов.

Архитектурно pipeline разделяет доменную логику и CLI-слой. Dataset-часть отвечает за конвертацию сырых схем, статистический аудит и намеренные изменения датасета по recipe; training-часть строит план обучения и запускает Ultralytics YOLO; reporting-часть формирует per-class AP; ONNX-часть выполняет экспорт, оптимизацию графа, калибровку INT8 и подготовку артефактов под CPU или CUDA runtime. Такое разделение важно для ВКР: оно делает подготовку textured ROI detector воспроизводимой и отделяет исследовательские решения о данных от инженерных решений о deployment. Поддержка ONNX-экспорта важна, поскольку Ultralytics рассматривает export mode как основной путь переноса обученной модели в форматы для deployment, включая ONNX [26, 27]. Логика pipeline представлена на рисунке 4.1.



Рисунок 4.1: Generic YOLO pipeline для детектора текстурированных ROI: конвертация и аудит датасетов, обучение, AP/confusion analysis, ONNX export, runtime optimization и benchmark/deploy этап.

После рисунка важно подчеркнуть границу ответственности pipeline. Он не является частью hot path IDet и не выполняется при обработке кадра. Его результат — проверенный набор обучающих данных, отчетов и deploy-артефактов, которые затем передаются в C++/ONNX Runtime контур. Благодаря этому IDet остается CPU-first библиотекой без зависимости от Python training framework, а обучение и экспериментальная проверка модели сохраняются как отдельный воспроизводимый процесс.

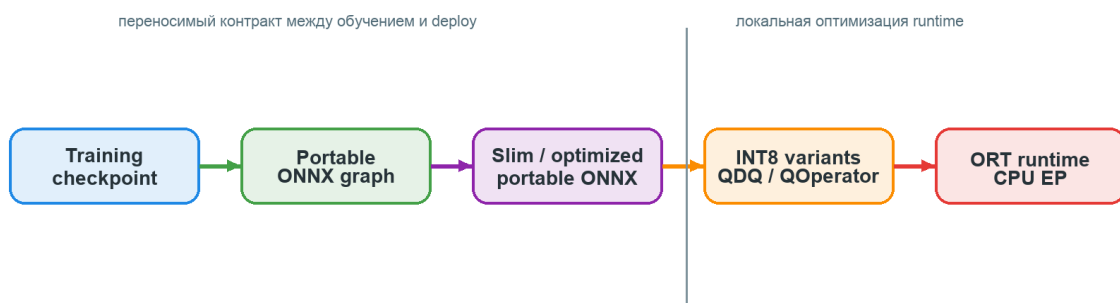


Рисунок 4.2: Путь подготовки deployable артефакта: от training checkpoint к переносимому ONNX и исполнению через ONNX Runtime CPU Execution Provider.

Использовались два исходных датасета: DeepFashion2 и Fashionpedia. DeepFashion2 содержит изображения одежды с аннотациями для detection, segmentation, landmarks и retrieval [30]. Fashionpedia построена вокруг fashion ontology и содержит категории одежды, части одежды и fine-grained attributes [31]. Оба датасета были переведены в YOLO-формат, после чего из исходных таксономий выделены 7 ключевых классов, согласованных между источниками.

4.3 Контракт PT-to-ONNX артефакта

Отдельным модулем generic YOLO pipeline является stage экспорта обученной PT-модели в ONNX. Этот этап нужен не для отчетности, а для замыкания цикла между обучением и IDet: библиотека не должна зависеть от Python training framework, поэтому в runtime передается стабильный ONNX-артефакт с известной формой входа, набором выходов и на-

стройками оптимизации. В ONNX Runtime такой артефакт может дополнительно проходить graph optimizations, включая устранение лишних узлов, fusion операций и оптимизации layout в зависимости от выбранного уровня [22].

Таблица 4.1: Контракт PT-to-ONNX артефакта для детектора текстурированных ROI

Элемент контракта	Требование
Входной артефакт	Лучший training checkpoint, полученный после обучения YOLO-модели на согласованной таксономии.
Экспортируемый формат	ONNX model artifact, совместимый с ONNX Runtime CPU Execution Provider.
Размер входа	Фиксированный или явно задокументированный dynamic input shape; размер должен совпадать с preprocessing в IDet.
Параметры экспорта	format=onnx, выбранный opset, режим dynamic, simplify, batch size и флаг встроенного NMS фиксируются в отчете pipeline.
Проверка корректности	Сравнение выходов PT и ONNX на калибровочных изображениях: shape, диапазоны confidence, число кандидатов и близость координат.
Проверка переносимости	Portable ONNX не должен содержать target-specific узлы ONNX Runtime layout optimization, например <code>com.microsoft.nchwc::Conv</code> ; такие файлы допустимы только как отдельные ORT-optimized artifacts для конкретной машины.
Оптимизация	Сохранение версии до оптимизации и версии после ORT graph optimization для воспроизводимого benchmark.
Метаданные	Версия pipeline, дата экспорта, dataset yaml, names/classes, hash весов и параметры preprocessing.
Интеграция	Артефакт регистрируется как texture adapter model и используется IDet без обращения к Python.

4.4 Единая 7-классовая таксономия

Целевая таксономия включает классы: `top_sleeved`, `outerwear`, `top_sleeveless`, `shorts`, `trousers`, `skirt`, `dress`. Выбор классов обусловлен задачей textured ROI: эти классы покрывают основные области одежды, которые чаще всего занимают значительную часть кадра и имеют воспринимаемую фактуру.

Смысл такой таксономии не в том, чтобы сохранить полную семантическую детализацию исходных fashion-датасетов. Для кодировщика важнее другой вопрос: есть ли в данной области визуально значимая фактура, которую нежелательно разрушать агрессивным квантованием. Поэтому близкие категории объединяются до уровня, достаточного для ROI-политики. Это снижает несовместимость DeepFashion2 и Fashionpedia, уменьшает число редких классов и делает выход модели стабильнее для downstream-решения о приоритете.

Таксономия также задает границу между ML-частью и видеокодированием. Модель может различать `dress`, `skirt` или `trousers`, но downstream-слой не обязан назначать каждому классу отдельный qoffset. В практической политике эти классы могут попадать в

одну группу textured ROI со средним приоритетом, ниже text и face. Поэтому 7-классовая схема используется как обучающая и диагностическая структура, а не как обязательная семантическая схема кодировщика.

4.5 Аудит датасетов после конвертации

В таблице 4.2 приведена сводка по трем конфигурациям: DeepFashion2, Fashionpedia и объединенному набору. Во всех случаях отсутствуют пустые разметочные записи, отсутствующие метки, «осиротевшие» метки и нечитаемые изображения. Это является важным инженерным результатом: качество модели начинается не с архитектуры, а с корректности подготовленного корпуса.

Аудит выполняет две функции. Во-первых, он подтверждает воспроизводимость training pipeline: каждый split содержит изображения, разметку и активные классы, а значит модель обучается на валидной структуре данных. Во-вторых, он задает границы интерпретации результатов. Если после конвертации меняются пропорции train/val/test или исчезает часть объектов, итоговая mAP уже не отражает качество исходного датасета. Поэтому до обучения важно проверять не только количество изображений, но и число instances, среднее число объектов на изображение, распределение классов, геометрию bbox и наличие некорректных строк разметки.

Объединенный набор заметно крупнее отдельных источников: он содержит 269559 изображений и 439399 объектов. Это повышает разнообразие сцен и снижает зависимость модели от одной аннотационной схемы, но одновременно усложняет контроль распределений. Поэтому объединение не рассматривается как механическое сложение датасетов: оно требует единой таксономии, проверки split-структуры и анализа того, не ухудшается ли качество редких классов после слияния.

Таблица 4.2: Сводная статистика датасетов после конвертации в YOLO-формат

Датасет	Images	Instances	Train img	Val img	Test img
DeepFashion2	224114	364675	179291	22411	22412
Fashionpedia	45445	74724	36356	4544	4545
Объединенный набор	269559	439399	234516	32347	2696

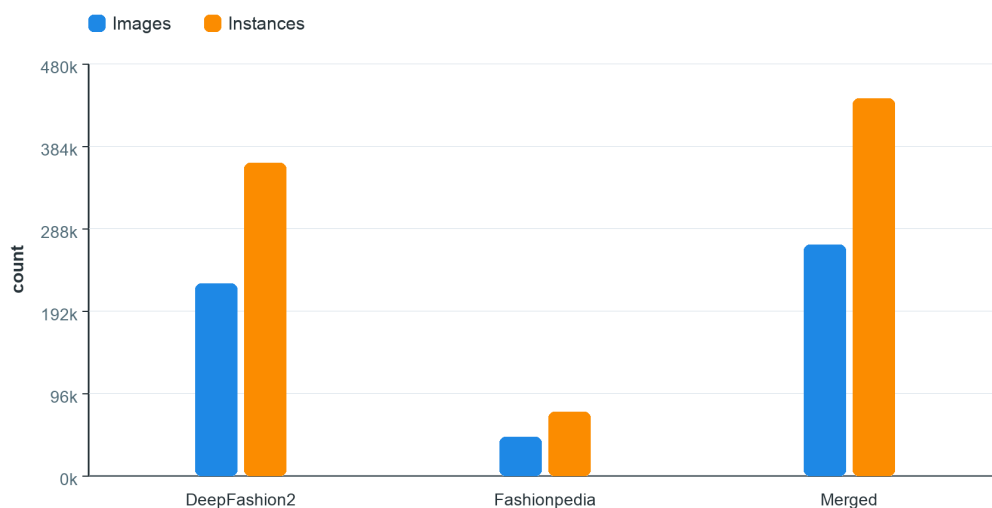


Рисунок 4.3: Сравнение масштаба датасетов после приведения к общей таксономии.

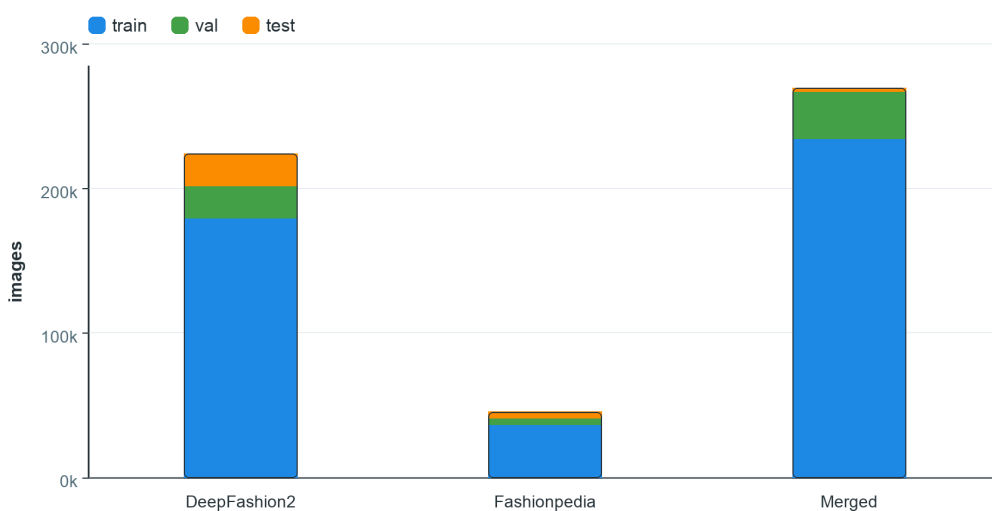


Рисунок 4.4: Распределение изображений по train/val/test-разбиениям для DeepFashion2, Fashionpedia и объединенного набора.

4.6 Дисбаланс классов

Класс `top_sleeved` доминирует во всех конфигурациях. В DeepFashion2 он составляет 126230 объектов, в Fashionpedia — 24803, а в объединенном наборе — 151033. При этом в Fashionpedia класс `top_sleeveless` содержит только 741 объект, что создает сильный imbalance. После объединения редкие классы получают больше примеров, а отношение крупнейшего класса к минимальному становится существенно мягче.

Дисбаланс влияет не только на итоговую mAP, но и на поведение модели в streaming-сценарии. Частые классы формируют большую часть градиента при обучении, поэтому модель может стать устойчивой к типичным верхним предметам одежды и менее уверенной на

редких силуэтах. Для ROI-задачи это означает риск систематического недо выделения некоторых визуально значимых областей. Именно поэтому в работе отдельно рассматриваются per-class AP и нормализованные матрицы ошибок: агрегированная метрика по всем классам может скрывать слабые места в редких категориях.

Объединенный набор уменьшает этот риск, но не устраняет его полностью. Например, `top_sleeveless` остается самым малым классом даже после объединения, поэтому его качество следует оценивать отдельно. В производственной политике это можно компенсировать не только обучением, но и downstream-логикой: если класс относится к `textured ROI`, то ошибка между близкими типами одежды менее опасна, чем ошибка `background/foreground`. Такой взгляд позволяет не переоптимизировать `fine-grained classification` там, где для кодировщика достаточно корректного покрытия области.

Таблица 4.3: Распределение объектов по классам в объединенном датасете

Класс	Train	Val	Test	Total
<code>top_sleeved</code>	132382	17116	1535	151033
<code>dress</code>	67265	9833	813	77911
<code>trousers</code>	67226	9631	844	77701
<code>shorts</code>	36958	6203	484	43645
<code>skirt</code>	35909	6163	493	42565
<code>outerwear</code>	20407	4556	325	25288
<code>top_sleeveless</code>	17033	3925	298	21256

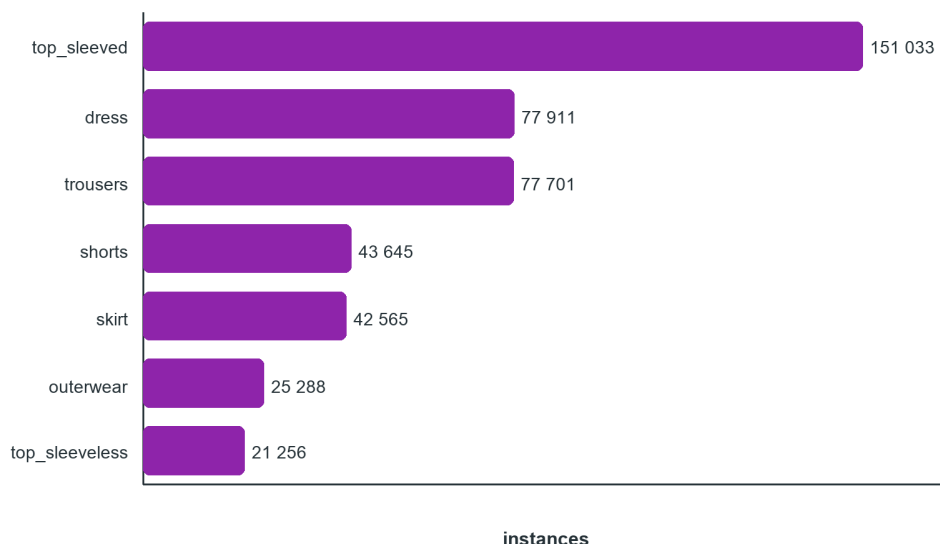


Рисунок 4.5: Распределение объектов по классам в объединенном датасете.

4.7 Геометрия изображений и `bbox`

Fashion-домены имеют выраженную вертикальную ориентацию. Для DeepFashion2 средний размер изображения составляет примерно 602.5×756.8 на `train split`, для

Fashionpedia — 755.3×986.1 , для merged — 628.3×795.7 . Центры bbox устойчиво находятся около центра кадра: для merged train средний bbox center равен $(0.5008, 0.5237)$, а средняя площадь bbox — 0.2709.

Это свойство имеет два следствия. С одной стороны, модель может переобучиться на центральное расположение объектов. Поэтому при обучении применялись современные аугментации, особенно геометрические: shift, scale, perspective/affine transforms и mosaic-like augmentations [29]. С другой стороны, центральность bbox полезна для реального streaming-контента с человеком в кадре: одежда ведущего часто действительно находится в центральной области, а значит textured ROI можно стабилизировать во времени.

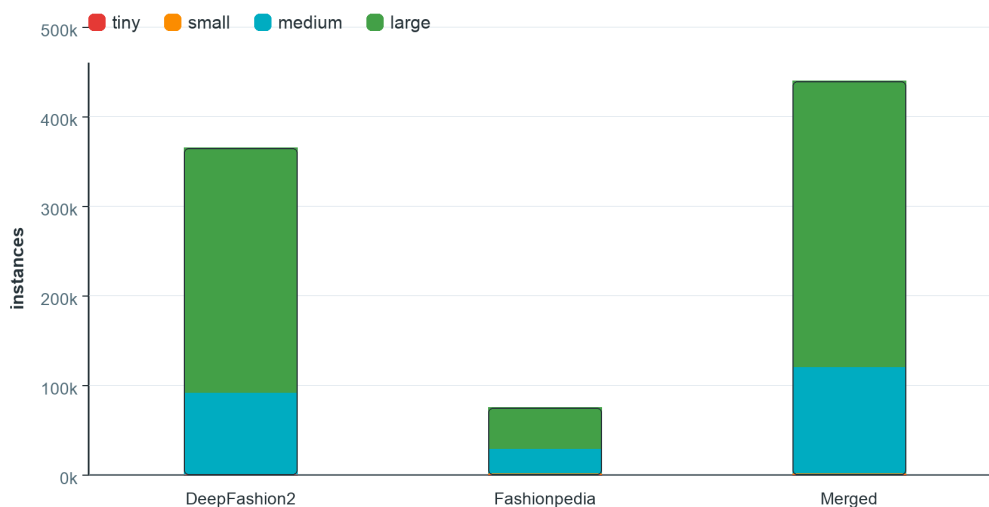


Рисунок 4.6: Распределение bbox по размерным группам после конвертации в YOLO-формат.

4.8 Режимы обучения

В работе рассматривались следующие режимы:

- 1) обучение только на DeepFashion2-7c;
- 2) обучение только на Fashionpedia-7c;
- 3) transfer learning: DeepFashion2 → Fashionpedia;
- 4) обучение на объединенном наборе DeepFashion2 и Fashionpedia;
- 5) сравнение YOLO26n и YOLO26s.

Итоговый выбор сделан в пользу YOLO26n на объединенном наборе. YOLO26s местами давал более высокие метрики, включая $mAP@0.5:0.95$ выше 0.84, но был тяжелее. Для ВКР важнее не абсолютный максимум mAP , а возможность интеграции в IDet с жестким CPU-бюджетом. Поэтому метрики обучения рассматриваются как первый уровень проверки; окончательный вывод о пригодности для real-time режима требует отдельного CPU-бенчмарка экспортированного ONNX-артефакта вместе с pre/post-processing.

4.9 Артефакты ONNX Runtime и квантизация

После обучения модель должна пройти отдельный этап проверки как deployable artifact. Для этого недостаточно сохранить лучший training checkpoint: необходимо убедиться, что экспортированный ONNX-граф совместим с ONNX Runtime CPU Execution Provider, не содержит непереносимых внутренних layout-операторов и сохраняет ожидаемые формы выходов. ONNX Runtime поддерживает несколько классов оптимизации графа и отдельные механизмы квантизации, включая QDQ-представление и QOperator-представление [22, 25]. Однако наличие INT8-артефакта само по себе не гарантирует ускорение: итоговая скорость зависит от kernels, структуры графа, overhead dequantization и целевой архитектуры CPU.

В рамках pipeline целесообразно сохранять следующие артефакты:

- 1) лучший training checkpoint;
- 2) исходный экспортированный ONNX-граф;
- 3) slimmed ONNX после удаления лишних узлов;
- 4) FP32 ONNX для baseline CPU inference;
- 5) QDQ INT8 ONNX для сравнения с FP32;
- 6) QOperator INT8 ONNX, если он поддерживается CPU EP и не ухудшает качество;
- 7) inspection report с количеством операторов и проверкой доменов;
- 8) benchmark report с p50/p90/p95/p99 для каждого артефакта.

Особое внимание нужно уделять операциям домена `com.microsoft.nchw`. Они могут появляться в результате внутренних оптимизаций ONNX Runtime для конкретной машины. Такой граф полезен как target-specific optimized artifact, но не должен подменять portable ONNX contract между training pipeline и IDet. В дипломе это различие важно, потому что воспроизводимый deploy artifact должен быть переносимым, а локальная оптимизация под Kunpeng или другой CPU должна описываться как отдельный этап.

Таблица 4.4: Классификация ONNX/ORT-артефактов в pipeline

Артефакт	Роль	Основная проверка
Raw ONNX	Базовый экспорт из training framework.	Shape inputs/outputs, opset, отсутствие ошибок загрузки.
Slimmed ONNX	Упрощенный переносимый граф.	Сохранение численной близости и структуры outputs.
FP32 ONNX	Baseline deploy model.	p99 latency, memory footprint, operator counts.
QDQ INT8 ONNX	Квантизованный граф с Quantize/DeQuantize узлами.	Quality regression, dequantization overhead, CPU kernels.
QOperator INT8 ONNX	Граф с квантизованными операторами.	Поддержка операторов CPU EP и отсутствие fallback на медленные пути.
ORT optimized artifact	Локально оптимизированный артефакт.	Привязка к runtime/CPU, недопустимость как portable contract.

В опубликованной сводке deploy-артефактов IDet для YOLO26 фиксируется отдельный, более прикладной срез качества: входной размер 640, dynamic spatial input, ONNX opset 20, ONNX Runtime CPU Execution Provider и percentile calibration с уровнем 99.98% для INT8-вариантов [40]. Эта сводка дополняет training summaries: она показывает не только mAP, precision и recall, но и размер deploy-модели, количество узлов графа и поведение FP32/INT8-представлений.

Таблица 4.5: Сводка deploy-артефактов YOLO26 для CPU Runtime

Семейство	Набор данных	Представление	mAP50-95	mAP50	P	R	Mem, MB
YOLO26n	DeepFashion2	FP32	0.8131	0.9343	0.8974	0.8767	9.3
YOLO26n	объединенный	FP32	0.8202	0.9395	0.9319	0.8621	9.3
YOLO26n	DeepFashion2	INT8 variants	0.7466–0.7492	0.8848–0.8863	0.8211–0.8261	0.9038–0.9058	2.7–2.9
YOLO26n	объединенный	INT8 variants	0.7421–0.7437	0.8723–0.8736	0.9373–0.9404	0.7978–0.7984	2.7–2.9
YOLO26s	DeepFashion2	FP32	0.8364	0.9490	0.9167	0.8940	36.4
YOLO26s	объединенный	FP32	0.8161	0.9225	0.8982	0.8620	36.4
YOLO26s	DeepFashion2	INT8 variants	0.8158–0.8167	0.9403–0.9414	0.9203–0.9256	0.8817–0.8826	9.6–9.8
YOLO26s	объединенный	INT8 variants	0.7707–0.7740	0.8968–0.9000	0.8980–0.9018	0.8465–0.8492	9.6–9.8

Из таблицы видно, что квантизация резко уменьшает размер артефакта: для YOLO26n примерно с 9.3 MB до 2.7–2.9 MB, для YOLO26s — с 36.4 MB до 9.6–9.8 MB. Однако это сопровождается заметной потерей mAP50-95, особенно для объединенного набора. Поэтому INT8-варианты в данной работе следует рассматривать как deploy-гипотезы, требующие отдельной проверки latency и качества, а не как автоматически лучший выбор. Для CPU-first IDet базовая FP32-модель YOLO26n остается сильной отправной точкой: она существенно меньше YOLO26s и сохраняет сопоставимые метрики в deploy-oriented сводке.

5. Экспериментальные результаты YOLO-моделей на объединенной таксономии

5.1 Граница экспериментальных выводов

В этой главе приведены результаты обучения и валидации YOLO26n как детектора текстурированных ROI. Эти результаты подтверждают качество object detection на подготовленной 7-классовой таксономии, но сами по себе не доказывают ускорение IDet и не доказывают рост качества видеокодирования. Для таких выводов нужны отдельные CPU-only прогоны ONNX Runtime с p50/p90/p95/p99 latency и отдельный эксперимент кодирования с одинаковым битрейтом. Такое разделение важно, потому что высокая mAP и низкая задержка полного ROI pipeline являются разными свойствами системы.

Поэтому численные выводы этой главы следует читать как характеристику ML-компонента, а не как финальный результат всей видеосистемы. Валидация YOLO отвечает на вопрос, способен ли детектор находить области одежды и фактуры на выбранной таксономии. Она не отвечает на вопросы о стоимости letterbox-преобразования в C++, о задержке NMS на CPU, о поведении ONNX Runtime после квантизации и о том, как кодировщик перераспределит биты по ROI. Эти уровни связаны, но должны измеряться разными экспериментами, чтобы работа не смешивала метрики обучения и метрики видеокодирования.

5.2 Аппаратная конфигурация

Обучение выполнялось на рабочей станции с NVIDIA RTX 4090 24GB, Intel Core i7-13700 и 64GB RAM. Такая конфигурация позволила запускать обучение YOLO26n/YOLO26s с тяжелыми аугментациями и большим merged dataset. RTX 4090 имеет 24 GB видеопамати, что делает ее практичной для обучения современных моделей компьютерного зрения [33].

Таблица 5.1: Аппаратная конфигурация обучения

Компонент	Значение
GPU	NVIDIA RTX 4090, 24 GB VRAM
CPU	Intel Core i7-13700
RAM	64 GB
Модели	YOLO26n, YOLO26s
Выбранная модель	YOLO26n
Критерий выбора	качество при минимальной вычислительной стоимости для CPU-first IDet

5.3 Итоговые метрики

Итоговые метрики YOLO26n на объединенной 7-классовой таксономии приведены в таблице 5.2. По результатам валидационного отчета модель достигла $mAP@0.5 = 0.938$ и

$mAP@0.5:0.95 = 0.818$ при $precision = 0.903$ и $recall = 0.873$. С точки зрения object detection это сильный результат, особенно если учитывать, что модель выбрана как легкий вариант для последующей CPU-интеграции. При этом в доступных отчетах нет подтвержденного значения оптимального F1-порога, поэтому он не используется как численный вывод.

Таблица 5.2: Итоговые метрики YOLO26n на объединенной 7-классовой таксономии

Метрика	Значение	Интерпретация
Precision	0.903	Модель сохраняет высокий уровень точности по валидационному отчету.
Recall	0.873	Модель находит большую часть релевантных областей одежды.
$mAP@0.5$	0.938	Высокое качество при стандартном IoU-пороге.
$mAP@0.5:0.95$	0.818	Устойчивое качество при строгих порогах локализации.

5.4 Поклассовая точность детектора

Поклассовая AP показывает, что модель наиболее уверенно распознает `trousers`, `dress`, `top_sleeved`, `shorts` и `skirt`. Более сложными являются `top_sleeveless` и `outerwear`. Это ожидаемо: `top_sleeveless` является менее представленным и визуально близким к другим верхним классам, а `outerwear` часто пересекается с `top_sleeved` и `dress`. Для поклассового сравнения в основном тексте достаточно графика: он показывает как мягкий $AP@0.5$, так и более строгий $AP@0.5:0.95$.

Для ROI-задачи особенно важно сравнивать $AP@0.5$ и $AP@0.5:0.95$ вместе. Если $AP@0.5$ высокий, а строгая AP заметно ниже, модель в целом находит объект, но геометрия `bbox` менее точна. В обычной классификации одежды это может быть вторичным, но для кодировщика геометрия напрямую задает защищаемую площадь. Поэтому классы с низкой строгой AP требуют downstream-ограничений: `clipping`, `area cap` или более консервативный `qoffset`. Такой анализ помогает не переоценивать качество модели по одной агрегированной mAP.

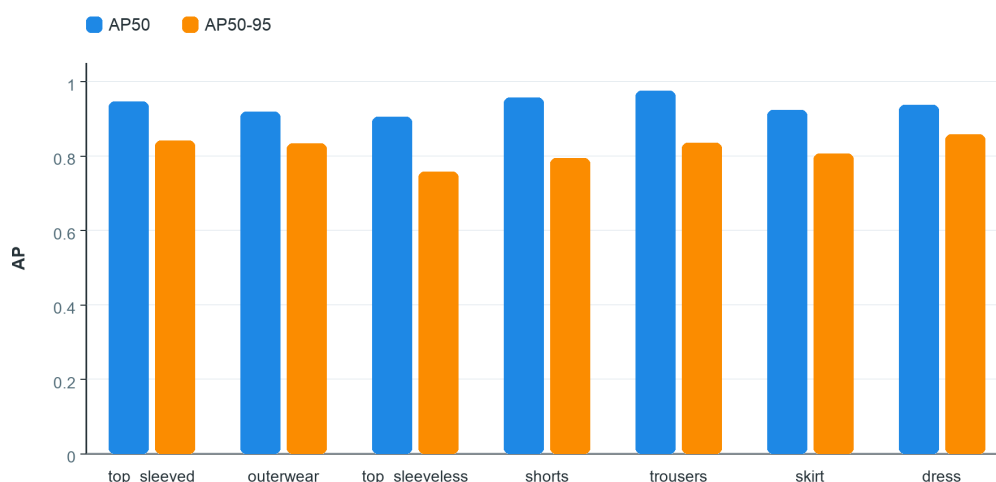


Рисунок 5.1: Поклассовая $AP@0.5$ для детектора текстурированных ROI.

5.5 Строгая поклассовая AP и сравнение моделей

Для ROI-задачи важна не только AP при IoU 0.5, но и устойчивость локализации при более строгих порогах. На рисунке 5.2 показана поклассовая $mAP@0.5:0.95$ для YOLO26n на объединенном наборе. Самым слабым классом остается `top_sleeveless` с $AP@0.5:0.95 = 0.758$; это согласуется с дисбалансом датасета и визуальной близостью sleeveless-вещей к другим верхним категориям. При этом `dress`, `top_sleeved`, `trousers` и `outerwear` сохраняют значения выше 0.83, что достаточно для роли ROI proposal detector.

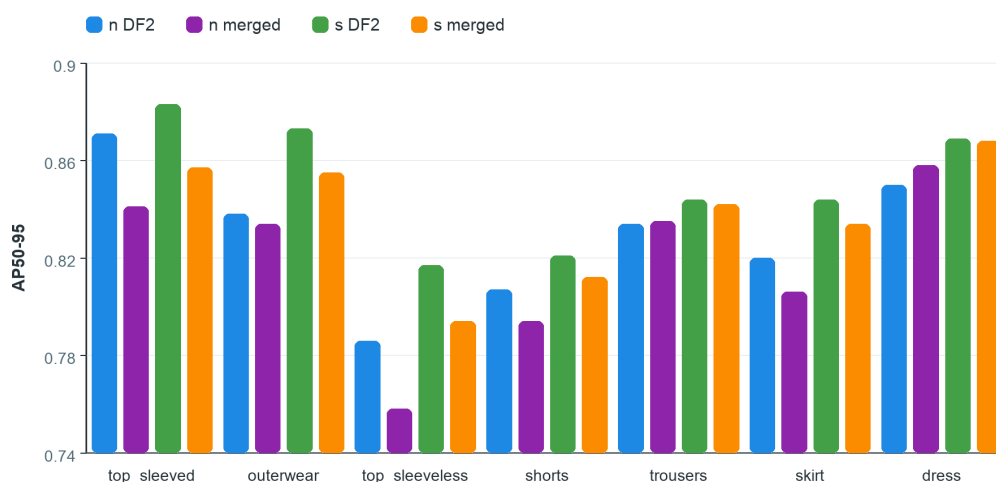


Рисунок 5.2: Поклассовая $AP@0.5:0.95$ для YOLO26n на объединенной 7-классовой таксономии.

Сравнение четырех запусков показывает ожидаемый компромисс. YOLO26s имеет около 9.47 млн параметров и 20.5 GFLOPs против 2.38 млн параметров и 5.2 GFLOPs у YOLO26n, то есть примерно четырехкратную вычислительную стоимость по валидационным отчетам. На DeepFashion2 это дает рост $mAP@0.5:0.95$ с 0.829 до 0.850. На объединенном датасете отчеты показывают 0.818 для YOLO26n и 0.837 для YOLO26s; это сравнение

нужно трактовать осторожно, потому что число validation images/instances в двух отчетах различается. Тем не менее общий вывод устойчив: более тяжелая модель дает умеренный прирост качества, но хуже соответствует CPU-first роли IDet.

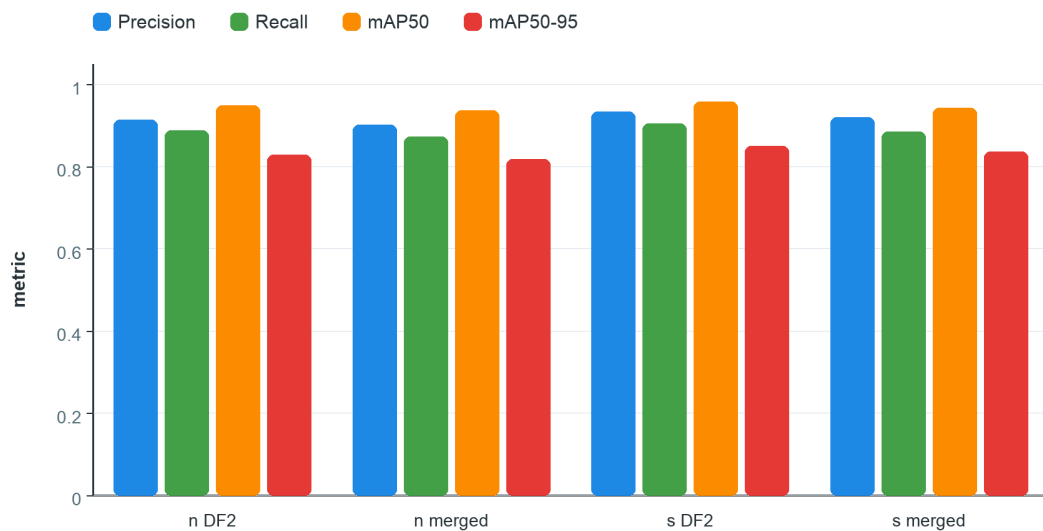


Рисунок 5.3: Сравнение precision, recall, mAP@0.5 и mAP@0.5:0.95 для YOLO26n/YOLO26s на сохраненных validation summaries.

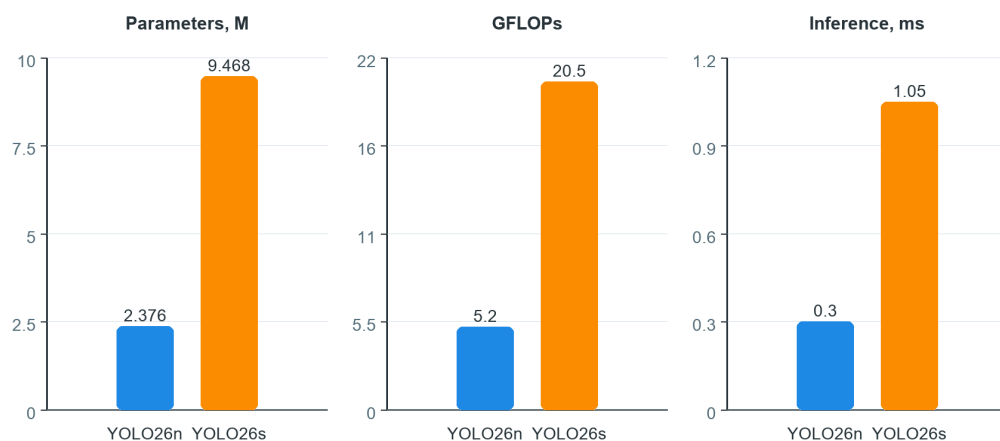


Рисунок 5.4: Сравнение параметров и GFLOPs для YOLO26n и YOLO26s.

5.6 Матрица ошибок

Нормализованная матрица ошибок показывает, что на диагонали модель имеет сильные значения: 0.92 для top_sleeved, 0.95 для trousers, 0.90 для shorts и 0.87 для dress. Более низкие значения у top_sleeveless (0.80) и outerwear (0.83) объясняются визуальной близостью классов и дисбалансом данных. Заметная часть ошибок связана не с заменой одного класса одежды другим, а с фоном: для top_sleeved, dress,

trousers и skirt в background-столбце видны значения 0.36, 0.15, 0.14 и 0.12 соответственно. Для ROI-задачи это означает, что часть одежды может не получить textured ROI, если пороги и area policy выбраны слишком агрессивно.

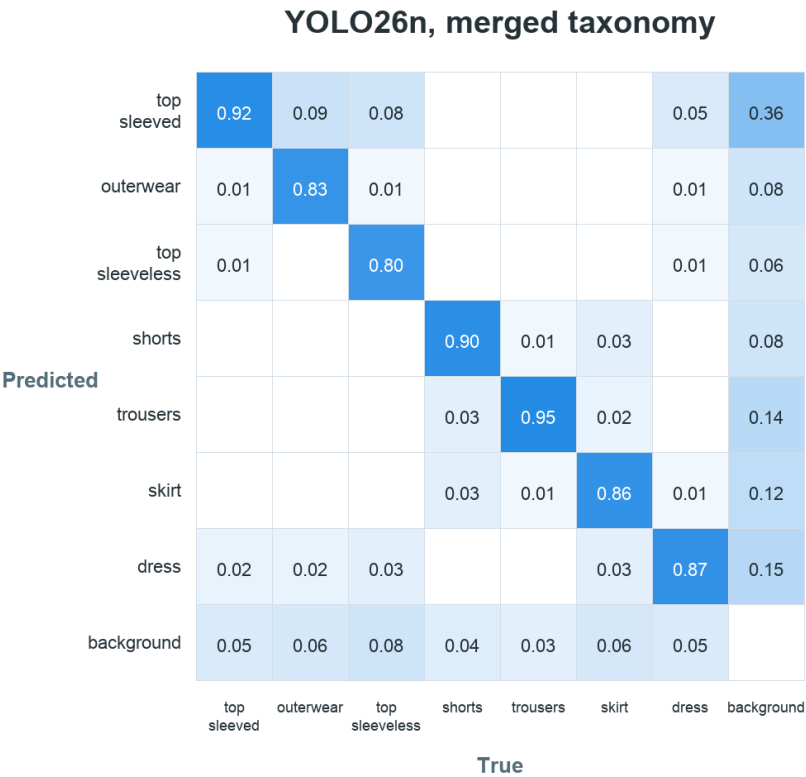


Рисунок 5.5: Нормализованная матрица ошибок YOLO26n на объединенной 7-классовой таксономии.

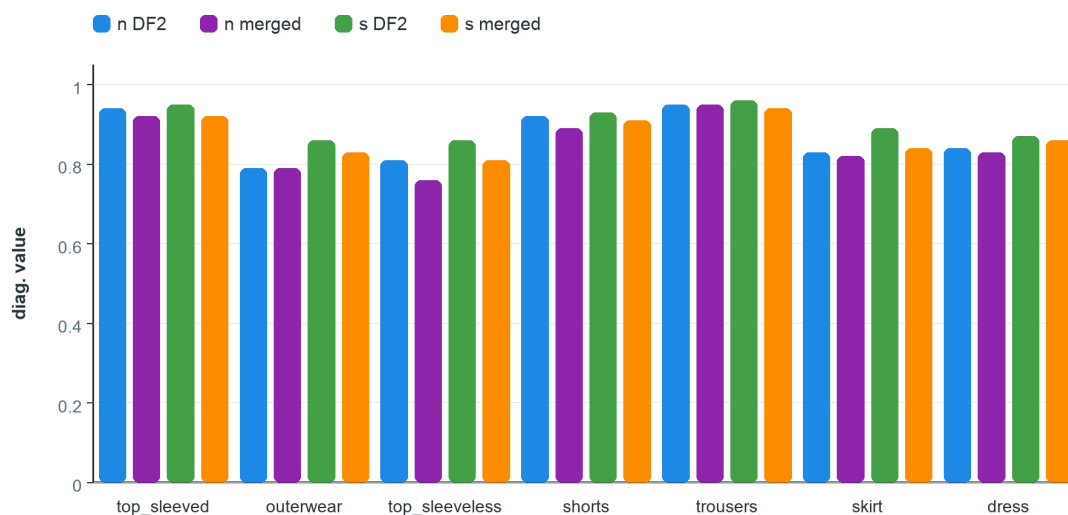


Рисунок 5.6: Сравнение диагональных значений нормализованных матриц ошибок для четырех YOLO-экспериментов.

Для задачи ROI proposal generation часть ошибок между близкими классами одежды менее критична, чем полный пропуск области: если модель все равно выделяет область одежды, она может быть защищена кодировщиком как textured ROI. Поэтому для интеграции в IDet важны не только class-level ошибки, но и геометрическое покрытие ROI относительно reference-разметки. Полные нормализованные матрицы для остальных запусков вынесены в приложение, чтобы не перегружать основной текст повторяющимися тепловыми картами.

5.7 Почему выбран YOLO26n

Финальное инженерное решение — выбрать YOLO26n, а не YOLO26s. YOLO26s имеет более высокую точность на части экспериментов, но требует около 20.5 GFLOPs против 5.2 GFLOPs у YOLO26n. Цель ВКР — не построить максимально точный fashion classifier, а получить модель, которую реалистично встроить в CPU-first IDet и затем проверить в режиме с ограниченным бюджетом на весь pre/post-processing. Поэтому YOLO26n является более обоснованным кандидатом для практической интеграции: он сохраняет $mAP@0.5:0.95 = 0.818$ на объединенном наборе и уменьшает вычислительную стоимость относительно YOLO26s примерно в четыре раза.

Этот выбор также подтверждается deploy-oriented сводкой IDet. При одинаковом входном размере и CPU Execution Provider FP32-артефакт YOLO26n для объединенного набора занимает 9.3 MB и показывает $mAP_{50-95} = 0.8202$, тогда как FP32-артефакт YOLO26s занимает 36.4 MB и в этой сводке не дает устойчивого преимущества на объединенном наборе. Это не отменяет того, что в отдельных training summaries YOLO26s может быть точнее; оно показывает другое: после перевода в deploy-контракт легкая модель выглядит более подходящей для CPU-first runtime.

Более тяжелая модель увеличивает не только время inference, но и давление на кэш, объем intermediate tensors и стоимость работы с памятью. В tiled-режиме это может уси-

ливать backend-bound поведение, уже наблюдаемое в профилировании. Поэтому выигрыш YOLO26s по точности нужно сопоставлять не с одним числом latency модели, а с полной стоимостью кадра: preprocessing, inference, post-processing, слияние ROI и влияние на p99. С этой точки зрения YOLO26n является более безопасной стартовой точкой для CPU-first системы, а INT8-варианты должны рассматриваться как отдельная линия дальнейшей проверки, потому что уменьшение memory footprint сопровождается измеримой потерей detection quality.

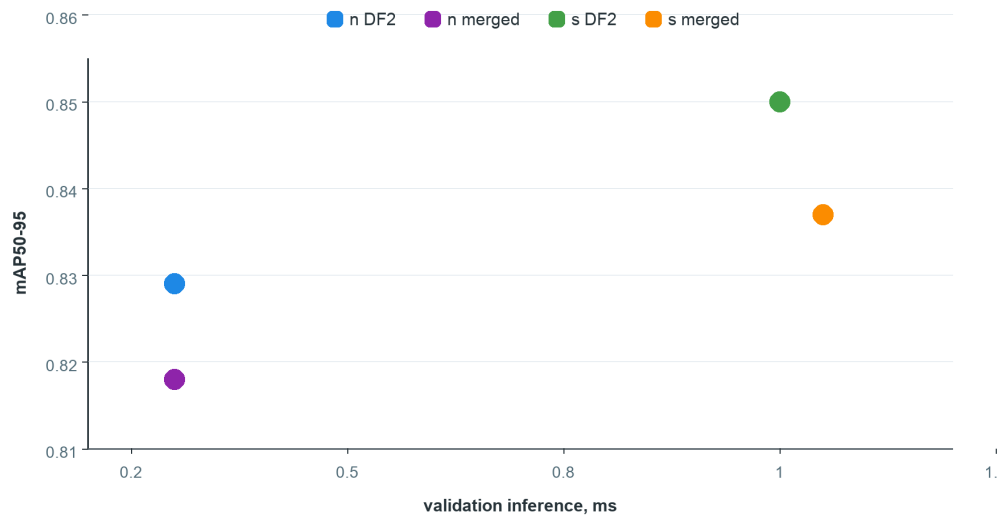


Рисунок 5.7: Компромисс между качеством детектора и вычислительной стоимостью по валидационным отчетам.

6. Интеграция ROI в видеокодирование

6.1 ROI как управляющий сигнал, а не конечный результат

В данной работе ROI-детектор не является финальным продуктом сам по себе. Его выход — это управляющий сигнал для кодировщика. Поэтому оценивать систему нужно не только по detection metrics, но и по тому, как ROI влияет на итоговое качество видео при фиксированном битрейте.

Пусть S — стратегия кодирования, D — ROI detector, E — encoder. Тогда полный конвейер можно записать как

$$\hat{V} = E(V, S(D(V))), \quad (6.1)$$

где V — исходное видео, $D(V)$ — последовательность ROI, S — политика приоритизации и mapping в qoffset, $\hat{V}$ — сжатый поток. Важно, что D не оптимизируется изолированно; он должен быть полезен внутри E .

6.2 Политика qoffset

Для каждой ROI задается приоритет p_i . Сильнее всего защищается текст, затем лица, затем textured ROI. Практическая политика может быть задана таблицей 6.1.

Таблица 6.1: Пример политики приоритизации ROI

Тип ROI	Priority	qoffset	Причина
Text ROI	1.00	-0.20	Читаемость текста критична для пользователя.
Face ROI	0.85	-0.15	Искажения лица заметны субъективно.
Textured ROI	0.55	-0.08	Фактура повышает perceived sharpness, но не должна вытеснять текст/лицо.
Background	0.00	0.00	Фон кодируется без дополнительной защиты.

Эти значения не являются универсальными константами; они задают начальную инженерную политику для экспериментов. В реальной системе qoffset следует подбирать на validation corpus с учетом битрейта, кодека и контента.

6.3 Fallback-режимы

Поддержка ROI metadata различается между кодировщиками и сборками FFmpeg/x265 [10, 12]. Поэтому в работе предусматриваются fallback-режимы:

- 1) использование native ROI metadata, если кодировщик поддерживает AVRegionOfInterest;
- 2) настройка adaptive quantization и rate-control параметров;
- 3) слабая предобработка фона вне ROI для снижения его локальной сложности;

- 4) offline selection режим, где ROI используется только для анализа качества и построения отчетов.

Наличие fallback-режимов важно для переносимости. В идеальном варианте IDet формирует ROI metadata, а кодировщик напрямую применяет qoffset к соответствующим областям. Однако в реальной инфраструктуре могут отличаться версии FFmpeg, параметры сборки, доступные кодеки и способ прокидывания side data между фильтрами. Поэтому ROI-aware система не должна зависеть от единственного backend-механизма. Если native ROI недоступна, система может перейти в диагностический режим, где ROI используются для измерения качества, либо в приближенный режим, где фон обрабатывается мягче, чем области интереса.

Такой подход не меняет исследовательскую гипотезу, но делает архитектуру практичной. Основной эксперимент качества должен по возможности использовать native ROI/qoffset путь, поскольку именно он проверяет идею перераспределения битового бюджета. Fallback-пути нужны как инженерная страховка: они позволяют сохранить унифицированный ROI extraction layer и продолжить анализ даже на окружениях, где конкретный кодировщик не поддерживает нужный способ передачи областей.

6.4 Целевой эксперимент качества

Финальная проверка должна сравнивать не только detection metrics, но и качество закодированного видео. Минимальная матрица экспериментов приведена в таблице 6.2.

Таблица 6.2: Матрица экспериментов ROI-aware кодирования

Конфигурация	Text	Face	Texture	Ожидаемый эффект
Baseline	нет	нет	нет	Равномерное кодирование без пространственного приоритета.
Text-only	да	нет	нет	Рост читаемости текста и интерфейса.
Face-only	нет	да	нет	Улучшение качества лиц.
Text+Face	да	да	нет	Базовый semantic ROI режим.
Full ROI	да	да	да	Защита текста, лиц и фактурированных областей.

Измеряться должны VMAF, SSIM, ROI-weighted SSIM/VMAF, bitrate deviation, latency, FPS и стабильность ROI. Такой протокол связывает научную часть с практическим продуктом.

7. Методика сравнения baseline и ROI-aware закодированного потока

7.1 Цель эксперимента

Финальная проверка IDet должна выходить за пределы детекционных и inference-метрик. Цель эксперимента состоит в том, чтобы сравнить обычный закодированный поток без ROI и поток, закодированный с учетом ROI, полученных IDet. Сравнение должно выполняться при одинаковом исходном видео, одинаковом кодировщике, одинаковом preset, одинаковом целевом битрейте и одинаковых ограничениях на задержку. В такой постановке различие в качестве можно связывать именно с ROI-aware политикой, а не с изменением параметров кодирования.

Рабочая гипотеза состоит в том, что ROI-aware кодирование должно улучшить качество визуально значимых областей — текста, лиц и фактурированных объектов — за счет перераспределения битов от менее важных областей кадра. При этом глобальные VMAF/SSIM могут измениться умеренно, потому что ROI занимают лишь часть изображения. Поэтому основное подтверждение гипотезы следует искать в ROI-взвешенных метриках и визуальном сравнении локальных областей.

7.2 Экспериментальные ветки

Минимальная матрица эксперимента приведена в таблице 7.1. Эта матрица задает фиксированные ветки сравнения и не содержит количественных выводов: численные метрики должны вноситься только после одинаковых прогонов кодировщика.

Таблица 7.1: Ветки сравнения baseline и ROI-aware потока

Ветка	Text	Face	Texture	ROI policy	Ожидаемый эффект
Baseline	нет	нет	нет	ROI off	Равномерное кодирование без пространственного приоритета.
Text-only	да	нет	нет	text qoffset	Повышение читаемости текста, субтитров и интерфейса.
Face-only	нет	да	нет	face qoffset	Снижение заметных артефактов на лицах.
Text+Face	да	да	нет	semantic ROI	Базовая semantic ROI-политика для видео с человеком и текстом.
Full ROI	да	да	да	text+face+texture	Максимальная локальная защита при контроле площади ROI.

7.3 Контроль переменных

Для корректности эксперимента все ветки должны отличаться только ROI-политикой. Необходимо зафиксировать: исходный ролик, разрешение, fps, codec, preset, GOP structure,

pixel format, target bitrate или CRF policy, версию FFmpeg/x265, параметры rate-control и способ расчета метрик. Если одновременно меняются ROI и preset кодировщика, вывод о полезности ROI становится некорректным.

Таблица 7.2: Контролируемые параметры эксперимента кодирования

Параметр	Требование
Исходное видео	Один и тот же lossless или high-quality reference для всех веток.
Кодировщик	Одна и та же версия FFmpeg/x265 или другого backend.
Битрейт/CRF	Фиксированная политика rate-control; отклонение битрейта фиксируется отдельно.
Preset/GOP	Не изменяются между baseline и ROI-aware ветками.
ROI policy	Единственный изменяемый фактор эксперимента.
Метрики	VMAF, SSIM, ROI-weighted VMAF/SSIM, bitrate deviation, IDet latency.
Повторы	Для нестабильных окружений требуется несколько прогонов или фиксация CPU affinity/NUMA state.

7.4 ROI-взвешенные метрики качества

Глобальная метрика может слабо реагировать на локальное улучшение в ROI. Поэтому для анализа нужно дополнительно считать ROI-взвешенную версию метрики. Пусть $q_t(j)$ — значение локальной метрики качества на блоке j кадра t , а $w_t(j)$ — вес блока. Тогда ROI-взвешенное качество может быть рассчитано как

$$Q_{\text{ROI},t} = \frac{\sum_{j=1}^{M_t} w_t(j) q_t(j)}{\sum_{j=1}^{M_t} w_t(j)}. \quad (7.1)$$

Вес можно задать через пересечение блока с ROI:

$$w_t(j) = 1 + \beta_{\text{text}} I_{\text{text}}(j) + \beta_{\text{face}} I_{\text{face}}(j) + \beta_{\text{texture}} I_{\text{texture}}(j), \quad (7.2)$$

где $I_{\text{kind}}(j)$ показывает, пересекается ли блок с ROI соответствующего типа. Значения β должны фиксироваться в протоколе эксперимента. Для текстовых областей вес обычно должен быть выше, чем для textured ROI, поскольку потеря читаемости текста является более критичной.

7.5 Форма фиксации итоговых метрик

Таблица 7.3 задает форму фиксации результатов будущей серии кодирования. Прочерк означает, что метрика не публикуется до завершения одинаковых прогонов всех веток; такие поля не должны интерпретироваться как нулевые значения или предварительные результаты.

Таблица 7.3: Форма сравнения качества baseline и ROI-aware потока

Режим	Bitrate dev.	VMAF	ROI-VMAF	SSIM	ROI-SSIM	IDet p99	Encoder FPS	E2E FPS	Интерпретация
Baseline	—	—	—	—	—	0	—	—	Равномерное кодирование без ROI.
Text-only	—	—	—	—	—	—	—	—	Ожидается рост читаемости текста.
Face-only	—	—	—	—	—	—	—	—	Ожидается улучшение лиц при умеренной площади ROI.
Text+Face	—	—	—	—	—	—	—	—	Ожидается лучший semantic ROI баланс для типового streaming-контента.
Full ROI	—	—	—	—	—	—	—	—	Ожидается максимальное локальное качество при контроле extra area.

7.6 Критерии подтверждения гипотезы

Гипотеза о пользе ROI-aware кодирования считается подтвержденной, если при фиксированном битрейте выполняются следующие условия:

- 1) ROI-weighted VMAF или ROI-weighted SSIM выше, чем у baseline;
- 2) bitrate deviation находится в допустимом диапазоне;
- 3) p99 latency IDet не нарушает целевой бюджет видеоконвейера;
- 4) визуальное сравнение ROI-кропов подтверждает улучшение текста, лиц или фактур;
- 5) extra area ROI не становится настолько большой, что система фактически защищает весь кадр.

Отрицательный или частично отрицательный результат также должен быть описан честно. Например, если ROI-aware режим увеличивает ROI-SSIM, но слишком сильно ухудшает фон или снижает FPS кодировщика, это означает необходимость более строгой area policy, другого qoffset mapping или temporal smoothing. На основании принципа ROI-aware распределения качества ожидается, что при фиксированном битрейте поток с ROI из IDet будет демонстрировать более высокое качество в визуально значимых областях по сравнению с равномерным кодированием. Основным ожидаемый эффект должен проявиться в ROI-взвешенных метриках, поскольку именно они учитывают локальное качество текста, лиц и фактурированных областей.

8. Экспериментальная методика и критерии качества

На рисунке 8.1 показан общий протокол оценки: все режимы сравниваются на одинаковых входных видео, при одинаковом битрейте и одинаковых настройках кодировщика; меняется только ROI-политика. Такой протокол нужен, чтобы не спутать эффект ROI с эффектом другого пресета кодирования.

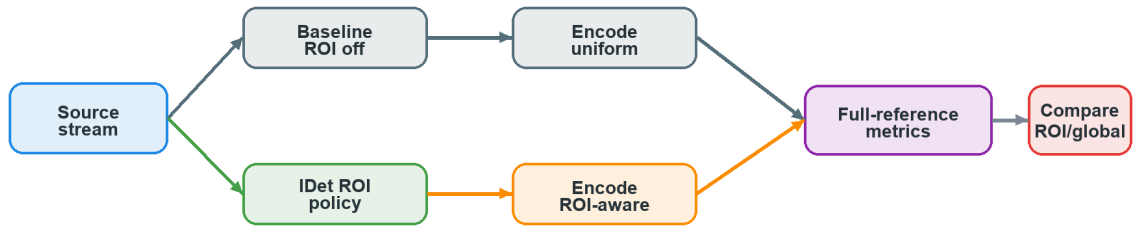


Рисунок 8.1: Экспериментальный протокол оценки ROI-aware кодирования.

8.1 Метрики качества видео

SSIM оценивает структурное сходство изображений и является классической full-reference метрикой [3]. В упрощенном виде SSIM для двух окон x и y записывается как

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}. \quad (8.1)$$

VMAF объединяет несколько признаков качества и лучше согласуется с субъективным восприятием в ряде видеосценариев [4]. В данной работе VMAF рассматривается как основная глобальная метрика, а ROI-weighted SSIM/VMAF — как дополнительная метрика для анализа локального улучшения.

PSNR также может использоваться как базовый контрольный показатель, поскольку он прост, воспроизводим и часто присутствует в инструментах оценки кодеков. Однако PSNR слабо отражает локальную воспринимаемость артефактов: одинаковое среднеквадратичное отклонение в фоне и в области текста имеет разную пользовательскую цену. Поэтому PSNR в данной работе не должен быть единственным критерием. Он полезен для sanity-check, но вывод о пользе ROI-aware кодирования должен опираться на SSIM/VMAF и их ROI-взвешенные варианты.

Ключевая методическая позиция состоит в том, что глобальная метрика может не вырасти существенно даже при полезном локальном эффекте. Если ROI занимает 10–20% кадра, улучшение в этих областях может быть частично компенсировано ухудшением фона. Это не является ошибкой постановки: цель ROI-aware кодирования состоит не в максимизации равномерного качества всего кадра, а в перераспределении качества в пользу областей с большей зрительной значимостью. Поэтому интерпретация метрик должна всегда сопровождаться анализом ROI-площади и локального качества.

8.2 Системные метрики

Так как цель работы — не только качество, но и deployability, обязательны системные метрики:

- 1) latency pre-processing;
- 2) latency inference;
- 3) latency post-processing;
- 4) суммарная задержка кадра;
- 5) p50/p90/p95/p99;
- 6) memory footprint;
- 7) FPS;
- 8) CPU utilization.

Инженерная целевая точка для IDet — ≤ 10 ms на кадр для всего ROI pipeline. Это соответствует примерно 100 FPS и оставляет некоторый бюджет для кодировщика и системной обвязки. Достижение этой точки должно проверяться отдельным benchmark-прогоном, а не выводиться из метрик обучения YOLO-модели.

Для near-real-time конвейера p99 важнее среднего значения. Средняя задержка может выглядеть приемлемой, но редкие пики приводят к нарушению deadline кадра, накоплению очереди или необходимости пропустить обработку части кадров. Поэтому в таблицах результатов p50 используется как характеристика типичного поведения, p95/p99 — как характеристика хвоста задержки, а итоговый выбор runtime-конфигурации должен учитывать именно хвост. Это особенно важно для tiled inference: разбиение кадра на плитки может уменьшить задержку одного режима, но усилить конкуренцию потоков и ухудшить стабильность.

Системные метрики также должны фиксировать условия измерения: число потоков ONNX Runtime, внешний параллелизм плиток, состояние I/O Binding, NUMA locality и режим вывода приложения. Измерение с отрисовкой или dump-логами нельзя смешивать с чистым latency benchmark, поскольку оно добавляет I/O и форматирование в критический путь. Поэтому экспериментальная методика разделяет режим измерения задержки и режим выгрузки ROI для area-based анализа.

8.3 Метрики покрытия ROI

Для оценки качества ROI в практическом pipeline недостаточно считать только число найденных областей. Конфигурация может вернуть много box-кандидатов, но при этом пропустить часть reference ROI или чрезмерно расширить защищаемую область. Поэтому для сравнения режимов IDet используется геометрическая оценка покрытия относительно reference-прогона. Reference boxes в этой постановке не являются ground truth-разметкой; это инженерный проху, полученный более дорогой или более консервативной конфигурацией.

Пусть R_{ref} — объединение reference ROI, R_{cand} — объединение candidate ROI, а $S(\cdot)$ — площадь множества пикселей. Тогда используются метрики:

$$\text{AreaRecall} = \frac{S(R_{\text{ref}} \cap R_{\text{cand}})}{S(R_{\text{ref}})}, \quad \text{AreaPrecision} = \frac{S(R_{\text{ref}} \cap R_{\text{cand}})}{S(R_{\text{cand}})}, \quad (8.2)$$

$$\text{AreaF}_1 = 2 \cdot \frac{\text{AreaRecall} \cdot \text{AreaPrecision}}{\text{AreaRecall} + \text{AreaPrecision}}, \quad (8.3)$$

$$\text{ExtraAreaRatio} = \frac{S(R_{\text{cand}}) - S(R_{\text{ref}} \cap R_{\text{cand}})}{S(R_{\text{ref}})}. \quad (8.4)$$

Метрика AreaRecall показывает, какая доля эталонной ROI-площади покрыта кандидатом, AreaPrecision штрафует за раздутые ROI, AreaF₁ балансирует покрытие и аккуратность, а ExtraAreaRatio ограничивает лишнюю защищаемую площадь. В финальных таблицах performance-quality конфигурации должны сортироваться по p99 latency по возрастанию, а качество должно проверяться по ограничениям на эти area-based метрики.

8.4 Дисциплина CPU-only benchmark

Для измерения IDet на CPU главным источником latency должны быть значения, которые печатает сам `idet_app`: `p50_ms`, `p90_ms`, `p95_ms` и `p99_ms`. Время внешнего Python subprocess допустимо сохранять только как служебное поле, но не как latency модели или pipeline. Для чистого benchmark необходимо отключать отрисовку, `dump` и подробный вывод: `--is_draw 0 --is_dump 0 --verbose 0`.

Финальные performance-выводы должны строиться только на release/perf-сборке без ASan/UBSan, поскольку санитайзеры меняют профиль памяти, `futex/wait` и распределение задержек. Для ARM/Kunpeng-прогонов дополнительно фиксируются число доступных CPU, NUMA-политика, параметры `threads_intra`, `threads_inter`, `tile_omp`, режим `cpu_placement`, `memory binding` и состояние automatic NUMA balancing. Если эти условия не зафиксированы, вывод о p99 latency следует считать ограниченным данным окружением.

8.5 NUMA, Kunpeng и ONNX Runtime CPU Execution Provider

Целевое CPU-only окружение требует отдельной дисциплины измерений. На серверных ARM-системах, таких как Kunpeng 920, итоговая задержка зависит не только от числа потоков, но и от размещения потоков по NUMA-узлам, локальности памяти и взаимодействия OpenMP с thread pool ONNX Runtime. ONNX Runtime предоставляет настройки `intra-op` и `inter-op threads`, а также рекомендации по настройке производительности и `threading` [23, 24]. Поэтому параметры `threads_intra`, `threads_inter` и `tile_omp` должны быть частью экспериментальной таблицы, а не скрытой настройкой окружения.

Automatic NUMA balancing в Linux может быть полезен для обычных приложений, но в pinned benchmark он способен добавлять шум за счет page-fault based migration и сканирования памяти [34]. Поэтому состояние `kernel.numa_balancing` необходимо фиксировать в протоколе. Если benchmark выполняется на ограниченном cuset, например толь-

ко на CPU 48–57, то сравнение конфигураций справедливо именно для этого cpuset. Изменение числа доступных CPU требует повторного grid search, поскольку оптимальные threads_intra и tile_omp могут измениться.

Для углубленного анализа hotspot-ов допустимо использовать CCA/perf profiling. Linux perf предоставляет механизм sampling и hardware performance counters [35]. На ARM-системах также может использоваться Statistical Profiling Extension, если она доступна ядру и firmware [36]. Если профилировщик выводит предупреждение о недоступности SPE, это не является ошибкой IDet, но ограничивает глубину анализа микроархитектурных событий.

Таблица 8.1: Минимальный протокол CPU-only performance-прогона

Параметр	Что фиксировать
Build profile	gcc-perf/clang-perf, отсутствие ASan/UBSan.
CPU topology	Модель CPU, число физических/логических ядер, доступный cpuset.
NUMA state	NUMA node, memory locality, состояние kernel.numa_balancing.
ORT settings	threads_intra, threads_inter, graph optimization level, I/O Binding.
Tiling settings	tiles_rc, tile_overlap, tile_omp, fixed tile input.
Benchmark mode	--is_draw 0 --is_dump 0 --verbose 0.
Quality dump mode	--is_draw 0 --is_dump 1 --verbose 0.
Latency metrics	p50, p90, p95, p99 из stdout idet_app.
Quality proxy	area recall, area precision, area F1, extra area ratio.

8.6 Кривые качества и вычислительной стоимости

Для каждой конфигурации можно построить quality-cost curve:

$$\mathcal{C}(S) = (L(S), Q_{\text{ROI}}(S), R(S)), \quad (8.5)$$

где L — задержка, Q_{ROI} — ROI-взвешенное качество, R — битрейт. Стратегия S_1 доминирует стратегию S_2 , если

$$Q_{\text{ROI}}(S_1) \geq Q_{\text{ROI}}(S_2), \quad L(S_1) \leq L(S_2), \quad R(S_1) \leq R(S_2), \quad (8.6)$$

и хотя бы одно неравенство строгое. Такой Pareto-анализ позволяет не сводить работу к одной метрике.

Для данной ВКР Pareto-анализ важен по двум причинам. Во-первых, модель с лучшей mAP может оказаться непригодной для CPU-only исполнения, если ее p99 latency выходит за пределы бюджета. Во-вторых, самая быстрая конфигурация может иметь плохую area precision и защищать слишком большую часть кадра. Поэтому каждая рабочая точка должна рассматриваться как тройка “задержка – качество ROI – расход битрейта/площади”, а не как одно число. В этом смысле графики latency-quality являются не иллюстрацией, а инструмен-

том выбора конфигурации.

Практически это означает, что итоговый выбор может зависеть от сценария. Для видеоконференции с жестким deadline предпочтительна speed-first точка с умеренным качеством покрытия; для offline-транскодирования можно выбрать более дорогую quality-balanced точку; для потока с большим количеством текста допустимо пожертвовать частью p99, если это заметно повышает читаемость. Такой подход делает методику гибкой и не вызывает архитектуру IDet к одному фиксированному профилю.

8.7 Абляционное исследование

Для доказательства полезности каждого компонента нужен ablation study. В частности, следует сравнить:

- 1) без ROI;
- 2) только text ROI;
- 3) только face ROI;
- 4) только textured ROI;
- 5) text + face;
- 6) text + face + textured;
- 7) разные пороги confidence для YOLO26n;
- 8) разные ограничения α на площадь ROI;
- 9) разные значения qoffset для textured ROI;
- 10) native ROI metadata против fallback preprocessing.

Такой протокол делает работу воспроизводимой и защищает ее от типичной слабости ML-проектов: наличия хорошей модели без понимания ее системного эффекта.

9. Экспериментальное исследование производительности IDet

9.1 Состав benchmark-эксперимента

Для оценки CPU-only поведения IDet использованы сводные таблицы grid-search эксперимента по трем режимам: text, face и textured ROI. Они содержат не только latency, но и area-based quality proxy относительно reference-прогона: AreaRecall, AreaPrecision, AreaF₁ и ExtraAreaRatio. Это принципиально важно: конфигурация не выбирается только по числу детекций или минимальному p50, потому что для кодировщика опасны как пропущенные ROI, так и чрезмерно раздутые области.

Во всех трех режимах чистый latency benchmark запускался с отключенными отрисовкой, сохранением диагностических областей и подробным выводом. Извлечение областей выполнялось отдельным коротким прогоном; эти режимы нельзя смешивать, иначе latency будет включать диагностическую запись результатов. В таблице 9.1 показаны две рабочие точки для каждого режима: speed-first и quality-balanced. Первая минимизирует p99 при выполнении базовых ограничений на coverage, вторая выбирает максимальный AreaF₁ из сводной таблицы и показывает цену качества в миллисекундах.

Таблица 9.1: Рабочие точки IDet по результатам grid-search эксперимента

Режим	Точка	p50	p95	p99	Area Recall	Area Precision	Area F1	Extra	Boxes	Tiles	Fixed HW	Threads	Intra	Inter	OMP	Desired
Text	speed-first	7.962	8.024	8.032	0.886	0.709	0.788	0.363	51	5x6	64x128	2/1/30	2	1	30	32
Text	quality-balanced	15.496	16.059	16.099	0.940	0.723	0.817	0.361	43	3x4	128x192	8/1/12	8	1	12	20
Face	speed-first	6.074	6.198	6.203	0.859	0.894	0.876	0.102	10	1x5	288x96	8/1/5	8	1	5	13
Face	quality-balanced	17.086	17.552	17.808	0.988	0.950	0.969	0.052	13	1x4	512x224	8/1/4	8	1	4	12
Textured	speed-first	9.018	9.163	9.230	0.920	0.827	0.871	0.192	37	2x4	128x128	8/1/8	8	1	8	16
Textured	quality-balanced	11.793	11.936	11.954	0.959	0.873	0.914	0.140	27	1x5	288x96	8/1/5	8	1	5	13

9.2 Latency-quality trade-off

На рисунке 9.1 каждая точка соответствует одной конфигурации grid search. Для всех трех режимов виден ожидаемый компромисс: повышение качества покрытия обычно требует большего p99, однако форма компромисса различается. Text ROI имеет быстрые конфигурации около 8 мс p99, но максимальный AreaF₁ достигается около 16 мс. Face ROI показывает самый быстрый режим около 5.4–6.2 мс p99, но quality-balanced конфигурации требуют уже 17–18 мс. Textured ROI оказывается наиболее удачным для текущего набора: speed-first точка уже дает AreaF₁ = 0.871 при p99=9.230 мс.



Рисунок 9.1: Компромисс между p99 latency и Area F1 для text, face и textured режимов IDet.

На рисунке 9.2 отдельно показаны p50/p95/p99 для speed-first конфигураций, прошедших ограничения на area-quality. Разница между p50 и p99 в этих точках невелика: например, для textured ROI p50=9.018 мс, p95=9.163 мс и p99=9.230 мс. Это важно для видеоконвейера: стабильный хвост задержки полезнее, чем один хороший средний показатель при редких длинных выбросах.

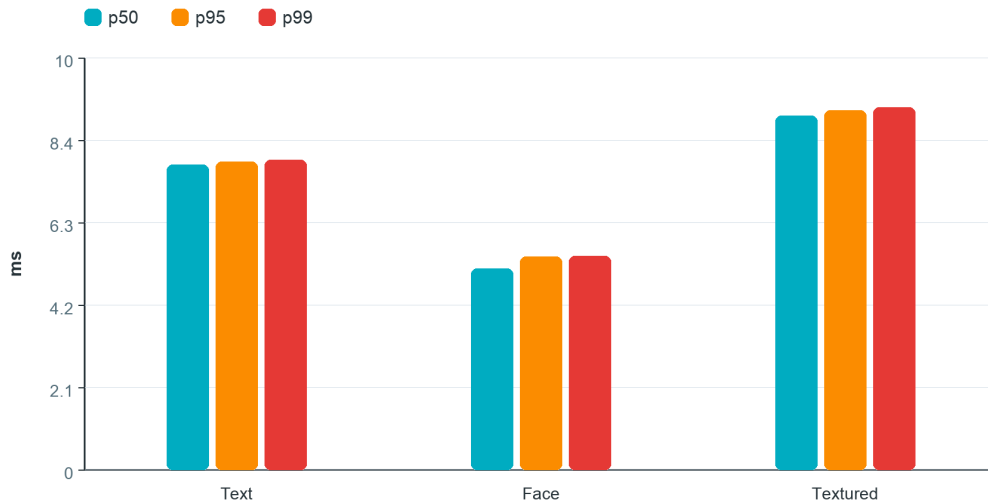


Рисунок 9.2: p50/p95/p99 для выбранных speed-first конфигураций text, face и textured режимов.

9.3 Интерпретация по режимам

Для text ROI наилучший быстрый вариант использует сетку 5x6, fixed input 64x128, threads_intra=2 и tile_omp=30. Он дает p99=8.032 мс и $\text{AreaF}_1 = 0.788$. Более качественная точка повышает area recall до 0.940 и AreaF_1 до 0.817, но p99 растет до 16.099 мс. Следовательно, text ROI имеет два практических режима: быстрый для жесткого latency budget и более дорогой для сценариев, где читаемость текста важнее хвоста задержки.

Для face ROI tiled-подход явно полезен в speed-first сценарии. В сводной таблице присутствуют single-shot точки для face; минимальный p99 single-shot составляет 14.008 мс при $\text{AreaF}_1 = 0.939$, тогда как tiled speed-first дает p99=6.203 мс при $\text{AreaF}_1 = 0.876$. Это показывает, что tiling снижает задержку, но может менять геометрию покрытия. Поэтому выбор face-конфигурации должен зависеть от того, что важнее в конкретном режиме: минимальный p99 или максимально точное покрытие лиц.

Для textured ROI лучшая быстрая точка уже удовлетворяет строгим требованиям качества: $\text{AreaRecall} = 0.920$, $\text{AreaPrecision} = 0.827$, $\text{AreaF}_1 = 0.871$ и $\text{ExtraAreaRatio} = 0.192$. Quality-balanced точка улучшает AreaF_1 до 0.914 и снижает лишнюю площадь до 0.140 при p99=11.954 мс. Для textured ROI это особенно важно, потому что чрезмерно широкие области одежды могут отнимать биты у текста и лица.

9.4 Profiling и I/O Binding

Профилирование использовалось как диагностический инструмент для выявления bottleneck, а не как источник финальных production latency. Сравнивались два режима: без I/O Binding и с I/O Binding при зафиксированном состоянии NUMA balancing. Результаты показывают, что I/O Binding существенно улучшает локальность памяти: доля *inner* traffic растет с 41.01% до 77.22% для textured single, с 48.01% до 94.31% для textured tiled, с 55.49% до 94.88% для face tiled и с 48.73% до 91.43% для text tiled. Это подтверждает инженерный

смысл binding: он не просто ускоряет отдельный вызов, а делает размещение памяти более предсказуемым в NUMA-сценарии.

Таблица 9.2: Влияние I/O Binding на локальность памяти

Режим	Inner with, %	Inner without, %	Прирост, п.п.	Total with, GB	Total without, GB
Textured single	77.22	41.01	36.21	15.82	35.75
Textured tiled	94.31	48.01	46.30	35.56	45.68
Face single	84.63	56.47	28.16	9.41	14.40
Face tiled	94.88	55.49	39.39	22.93	20.77
Text single	52.35	44.87	7.48	31.72	40.24
Text tiled	91.43	48.73	42.70	23.65	25.94

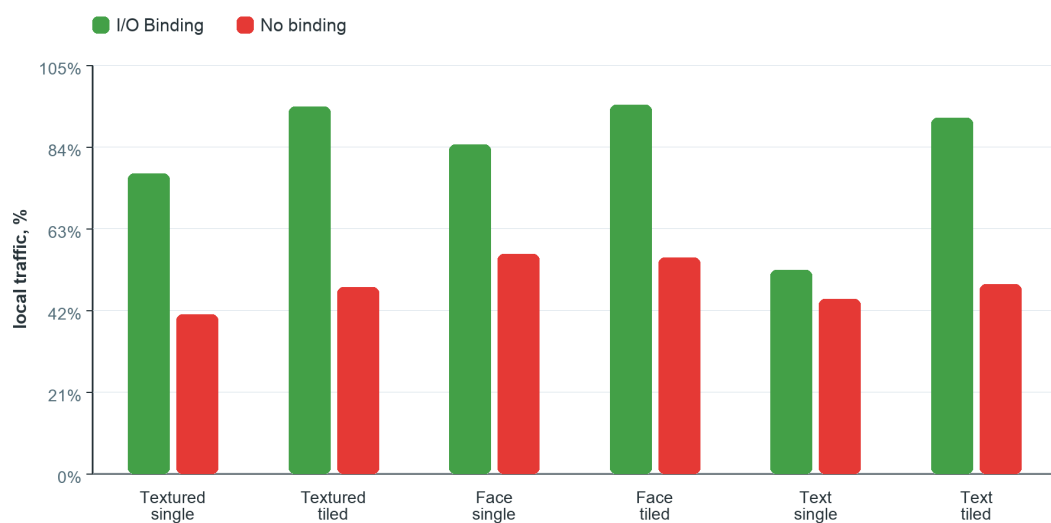


Рисунок 9.3: Доля локального memory traffic при включенном и выключенном I/O Binding.

Интерпретировать profiling нужно аккуратно. В большинстве режимов total memory traffic с binding ниже, но для face tiled он увеличивается с 20.77 GB до 22.93 GB. Следовательно, корректный вывод состоит не в том, что binding всегда уменьшает общий объем трафика, а в том, что он радикально повышает локальность обращений. Для pinned CPU benchmark это важнее: локальные обращения уменьшают зависимость от межсокетного трафика и делают p95/p99 более управляемыми.

9.5 Top-Down Microarchitecture Analysis

Top-Down Microarchitecture Analysis классифицирует, как процессор расходует pipeline slots [37, 38]. В этой модели *Retiring* означает долю полезно завершенных инструкций, *Frontend Bound* — ограничения подачи инструкций, *Backend Bound* — ожидание данных, памяти или execution resources, а *Bad Speculation* — потери из-за неверных предсказаний и сбросов pipeline.

На рисунке 9.4 показана реконструированная сводка для диагностических прогонов без I/O Binding. Single-shot режимы имеют более высокую долю Retiring: примерно 60.65%

для text, 62.77% для face и 65.88% для textured. В tiled-режиме доля Backend Bound возрастает: для text — до 51.94%, для face — до 49.47%, для textured — до 58.60%. Это означает, что tiled inference повышает параллелизм и может лучше использовать векторные участки, но одновременно усиливает роль памяти, синхронизации и runtime scheduling.

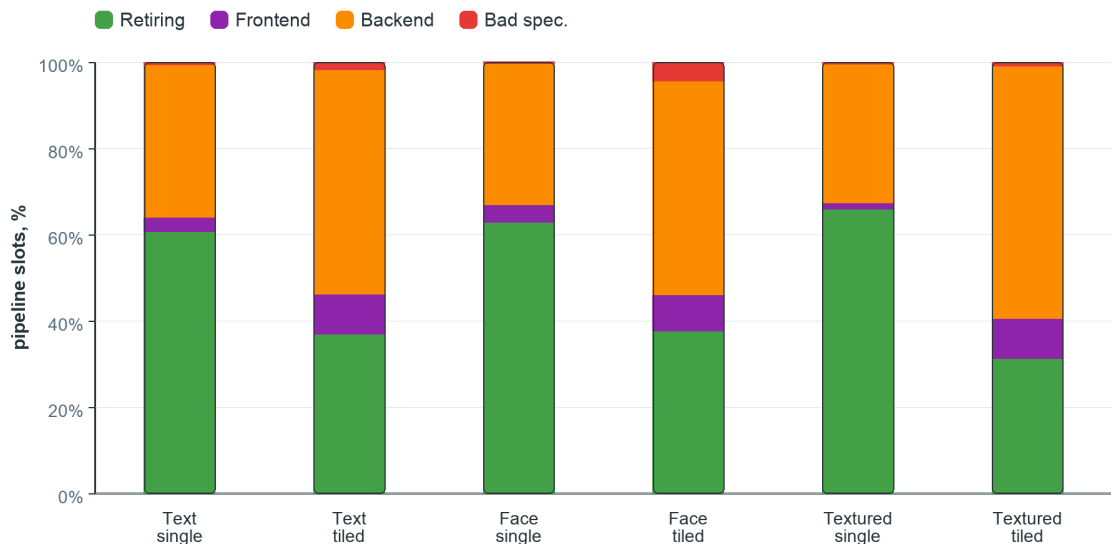


Рисунок 9.4: Top-Down Microarchitecture Analysis для single/tiled режимов в диагностических прогонах без I/O Binding.

Отдельно важно, что tiled-режимы резко увеличивают долю NEON SIMD instructions: для text ROI примерно с 10–14% до 40%, для face ROI примерно с 9–14% до 33–34%, для textured ROI примерно с 6–8% до 36–37%. При этом рост SIMD-доли не означает автоматическое снижение p99: если tiled-конфигурация становится backend-bound, ее итоговая задержка определяется не только арифметикой, но и доступом к памяти, stitching, NMS и scheduling. Поэтому выбор tiled-конфигурации должен оцениваться по p99 и area-quality одновременно, а не только по факту роста SIMD-доли.

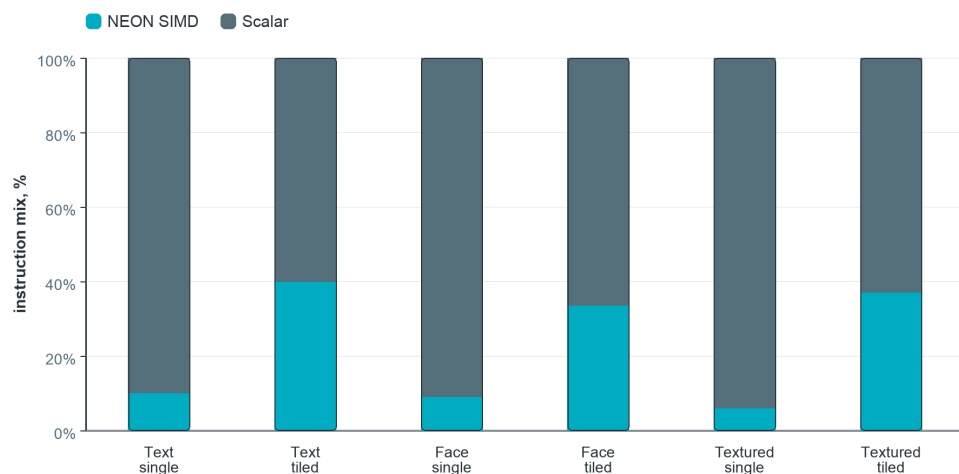


Рисунок 9.5: Доля NEON SIMD instructions в single и tiled режимах.

9.6 Ограничения текущих benchmark-данных

Полученные benchmark-таблицы являются сильным экспериментальным основанием для выбора конфигураций IDet, но их нельзя отождествлять с финальным end-to-end качеством видеостриминга. Во-первых, area-based метрики сравнивают candidate boxes с reference-прогоном, а не с ручной разметкой видео. Во-вторых, benchmark выполнен на отдельных изображениях, а не на видеопоследовательностях с motion blur и межкадровой нестабильностью. В-третьих, latency измеряет ROI detector layer, но не включает полный кодировщик и расчет VMAF/SSIM. Поэтому результаты этой главы подтверждают deployability отдельных режимов IDet и помогают выбрать runtime-конфигурации, а итоговая гипотеза о росте качества потока должна проверяться отдельным baseline vs ROI-aware encoding experiment.

10. Инженерные ограничения, риски и практическая значимость

10.1 Задержка как главный ограничитель

Главное отличие данной работы от обычного обучения YOLO-модели состоит в том, что модель должна жить внутри real-time pipeline. Если детектор текстурированных ROI улучшает mAP, но добавляет задержку, несовместимую с целевым режимом, он не подходит для архитектуры IDet. Поэтому YOLO26n выбран как компромиссный кандидат. Слабая модель может пропускать важные области, слишком тяжелая — разрушить latency budget. Правильный выбор находится на границе качества и скорости и должен подтверждаться CPU-only benchmark, а не только метриками обучения.

Задержка в IDet складывается из нескольких компонентов: подготовки входа, inference, декодирования выходов, NMS, обратного отображения координат и слияния ROI. Для нейросетевого детектора не всегда доминирует сама модель: на маленьких входных размерах заметными становятся преобразование layout, копирование буферов, сортировка кандидатов и синхронизация потоков. Поэтому оптимизация должна рассматривать весь hot path, а не только ONNX Runtime session. Именно этим объясняется внимание к I/O Binding, NUMA locality, tile-level parallelism и отсутствию лишних аллокаций.

10.2 Дисбаланс классов

DeepFashion2 и Fashionpedia имеют выраженный дисбаланс классов. В Fashionpedia-7с отношение крупнейшего класса к минимальному превышает 30х, а после объединения снижается примерно до 7х. Это объясняет, почему merged dataset предпочтителен для финальной модели. Но даже в merged dataset редкие классы требуют анализа per-class AP и confusion matrix, иначе слабые места будут скрыты общей mAP.

В производственной ROI-задаче дисбаланс опасен не только снижением качества редких классов, но и смещением confidence calibration. Если модель редко видит некоторый тип одежды, она может выдавать более низкую уверенность даже при корректной локализации. При жестком threshold такая область будет отброшена, хотя для кодировщика она могла быть полезной textured ROI. Поэтому threshold должен выбираться не только по максимальному F1 на validation split, но и с учетом area recall, extra area ratio и downstream-приоритета.

10.3 Центральное смещение объектов

На всех трех датасетах bbox centers концентрируются около центра кадра. Это естественно для fashion-изображений, но опасно для обобщения. Поэтому применялись геометрические аугментации. Для будущей версии системы полезно добавить тесты на нестандартных композициях: человек сбоку, крупный план ткани, движение камеры, частичное перекрытие одежды, несколько людей.

Центральное смещение может дать модели ложное упрощение задачи: вместо устой-

чивых признаков одежды она частично опирается на типичное положение объекта. В видеостриминге это не всегда работает. Говорящая голова действительно часто находится в центре, но product review, спортивная трансляция, вертикальные shorts-видео и видеоконференция с несколькими участниками нарушают эту предпосылку. Поэтому финальный validation corpus должен включать off-center сцены и кадры с несколькими конкурирующими ROI.

10.4 Риск ложных textured ROI

Детектор текстурированных ROI может выделять визуально сложный фон: ковры, стены, мебель, шумные поверхности. Это не всегда плохо, но может перераспределять биты в ненужные зоны. Поэтому textured ROI должен иметь меньший приоритет, чем text и face, а также ограничиваться budget cap по площади. Дополнительно можно использовать temporal persistence: область считается важной, если она сохраняется несколько кадров.

Ложноположительные textured ROI имеют иной характер риска, чем ложноположительные лица или текст. Если ошибочно выделен фон, пользователь может почти не заметить локального улучшения, но кодировщик уже потратит часть битового бюджета. Поэтому для textured режима особенно важны area precision и extra area ratio. Система должна предпочитать такие ошибки, при которых лишняя область мала и не пересекается с более приоритетными ROI. Если же textured ROI конфликтует с text ROI, приоритет текста должен сохраняться независимо от confidence детектора одежды.

10.5 Практическая значимость

Практическая ценность работы состоит в создании архитектуры, которую можно постепенно довести до production-ready состояния. Уже существуют:

- 1) IDet как CPU-oriented C++ база для детекции ROI;
- 2) generic YOLO training pipeline для подготовки и обучения моделей;
- 3) обученная YOLO26n модель textured ROI на объединенном наборе DeepFashion2 и Fashionpedia;
- 4) воспроизводимые dataset audits;
- 5) экспериментальный план для оценки quality-cost trade-off.

Дальнейшие шаги включают расширение CPU benchmark на видеопоследовательности, проверку нескольких ONNX/ORT/INT8 артефактов, end-to-end тестирование с кодировщиком и сравнение качества кодирования с ROI и без ROI.

11. Производственная интеграция и план развития IDet

11.1 Почему детектор текстурированных ROI должен быть частью IDet

Отдельный скрипт обучения или инференса может показать хорошие метрики, но для ВКР этого недостаточно. Ценность textured ROI появляется только тогда, когда модель становится частью системного конвейера: получает кадры из видеопотока, работает в предсказуемом временном бюджете, возвращает унифицированные ROI, не ломает память процесса и может быть включена или отключена конфигурацией. Поэтому в данной работе детектор текстурированных ROI рассматривается как третий режим IDet на этапе производственной интеграции, а не как отдельный ML-эксперимент.

С инженерной точки зрения это означает, что модель должна быть приведена к тому же публичному контракту, что text и face режимы IDet. На входе она получает кадр или плитки кадра, внутри движка выполняет letterbox, ONNX Runtime inference, декодирование YOLO outputs и NMS, а наружу через `Detector::detect` возвращает `VecQuad`. Дальше все различия между моделями скрываются внутри движка. Такой подход позволяет не переписывать downstream-интеграцию с кодировщиком при добавлении новых моделей.

11.2 План интеграции YOLO26n для текстурированных ROI

Полная интеграция модели в IDet может быть выполнена поэтапно. Первый этап — экспорт обученной модели в ONNX и проверка численного соответствия outputs между training framework и ONNX Runtime. Второй этап — реализация препроцессинга в C++: letterbox, нормализация, преобразование layout, подготовка входного буфера. Третий этап — post-processing: декодирование выходов, thresholding, NMS и преобразование координат из letterbox-пространства обратно в координаты исходного кадра. Четвертый этап — подключение результата к общему ROI fusion.

Ключевое требование состоит в том, чтобы pre/post-processing не оказался дороже самой нейросети. В object detection это типичная проблема: модель может выполняться быстро, но декодирование большого числа кандидатов, сортировка и NMS создают существенную задержку. Поэтому post-processing должен быть ограничен по числу кандидатов, использовать compact data structures и избегать лишних аллокаций. Для CPU-first реализации важно заранее выделять буферы под входы, выходы и промежуточные массивы.

```
1 idet::RuntimePolicy policy{};
2 policy.ort_intra_threads = 8;
3 policy.ort_inter_threads = 1;
4 policy.tile_omp_threads = 8;
5 policy.suppress_opencv = true;
6
7 auto cfg = idet::DetectorConfig::setup(idet::Task::Cloth,
    model_path);
```

```

8  cfg.runtime = policy;
9  cfg.infer.box_thresh = 0.25f;
10 cfg.infer.nms_iou = 0.5f;
11 cfg.infer.use_fast_iou = true;
12 cfg.infer.bind_io = true;
13 cfg.infer.fixed_input_dim = {640, 640};
14 cfg.infer.tiles_dim = {2, 4};
15
16 auto detector_result = idet::Detector::create(cfg);
17 if (!detector_result.ok()) {
18     // detector_result.status().message
19 }
20 auto detector = std::move(detector_result).value();
21
22 auto detections = detector.detect(image);
23 if (!detections.ok()) {
24     // detections.status().message
25 }
26 const idet::VecQuad& quads = detections.value();

```

Листинг 11.1: Сокращенный пример подключения cloth/YOLO режима через публичный API IDet

11.3 Политика потоков при производственной интеграции

В IDet необходимо разделить два уровня параллелизма. Первый уровень — внешний: параллельная обработка кадров или плиток. Второй уровень — внутренний: потоки ONNX Runtime внутри inference session. Если оба уровня включены без контроля, возникает oversubscription: слишком много потоков конкурируют за одни и те же ядра CPU, что ухудшает p95/p99 latency. Поэтому библиотека должна явно задавать число потоков для preprocessing, inference и post-processing.

Практический принцип состоит в следующем: для одного кадра с тремя ROI-режимами не нужно всегда запускать все модели одновременно. Если кадр похож на предыдущий, детектор текстурированных ROI можно вызывать раз в несколько кадров, а промежуточные ROI переносить через tracking/smoothing. Text ROI и face ROI также могут иметь разные частоты запуска. Это превращает систему из статической цепочки моделей в адаптивный конвейер, где тяжелые компоненты включаются с контролируемой частотой.

11.4 Профиль памяти и отсутствие динамических аллокаций в критическом пути

Для облачного сервиса важна не только средняя задержка, но и стабильность задержки. Один из источников нестабильности — динамические аллокации памяти в горячем пути. Поэтому IDet должен стремиться к zero-allocation hot path: все крупные буферы выделяются заранее, а при обработке кадра переиспользуются. Это касается входных изображений,

ONNX input tensors, выходных tensors, списков кандидатов, временных буферов NMS и результирующих ROI.

Если обозначить число кандидатов после декодирования как K , а число итоговых ROI как N , то практическая сложность post-processing должна быть ограничена сверху. Наивный NMS имеет сложность $O(K^2)$, что опасно при большом числе кандидатов. Поэтому в реализации необходимо применять предварительную фильтрацию по confidence и top-k selection:

$$K' = \min(K_{conf}, K_{max}), \quad K' \ll K. \quad (11.1)$$

После этого NMS выполняется по K' , а не по всем сырым кандидатам.

11.5 План CPU-бенчмарка

Для доказательства практической применимости textured ROI режима необходимо провести отдельный CPU benchmark. Он должен измерять не только среднее время, но и распределение задержек:

$$L_{total} = L_{pre} + L_{infer} + L_{post} + L_{fusion}. \quad (11.2)$$

В отчете должны быть p50, p90, p95 и p99. Среднее значение может скрывать редкие пики, которые в real-time pipeline приводят к пропускам кадров. Целевой критерий данной работы — приблизиться к $L_{total} \leq 10$ ms на кадр для выбранной конфигурации CPU. Если полный трехрежимный pipeline не укладывается в этот бюджет на каждом кадре, применяется sampling strategy: textured ROI вычисляется не на каждом кадре, а через заданный интервал, а между вызовами области стабилизируются.

11.6 Связь с качеством видеокодирования

Даже идеальная ROI-детекция не гарантирует улучшения качества, если кодировщик не использует ROI корректно. Поэтому итоговая проверка должна включать два уровня. Первый уровень — detection validation: mAP, precision, recall, confusion matrix. Второй уровень — video coding validation: сравнение закодированных потоков при одинаковом битрейте. Только второй уровень отвечает на вопрос ВКР: действительно ли ROI-aware подход улучшает облачный видеостриминг.

Возможен случай, когда textured ROI увеличивает качество одежды, но ухудшает фон. Это допустимо, если итоговое субъективное качество становится выше. Однако возможен и обратный случай: слишком много textured ROI заставляет кодировщик ухудшить лицо или текст. Поэтому textured ROI должен иметь меньший приоритет, чем face/text, и жесткое ограничение площади. В производственной системе параметры приоритетов должны подбираться по validation corpus.

11.7 Режимы эксплуатации

Предлагаемая система может работать в трех эксплуатационных режимах. Первый режим — *offline analysis*: ROI используются только для оценки качества и построения отчетов. Второй режим — *assisted encoding*: ROI передаются кодировщику при *offline/nearline* транскодировании, где допускается большая задержка. Третий режим — *real-time streaming*: ROI вычисляются в жестком *latency budget* и используются в *live/near-live* кодировании. Для ВКР наиболее важен третий режим, но первые два полезны как этапы валидации.

Таблица 11.1: Режимы эксплуатации ROI-aware системы

Режим	Цель	Ограничения	Роль textured ROI
Offline analysis	Проверка качества и отчетность	Нет жесткого latency	Анализ влияния одежды/фактуры на метрики.
Assisted encoding	Улучшение VoD/nearline транскодирования	Умеренный latency	Полноценное ROI metadata для кодировщика.
Real-time streaming	Live/interactive pipeline	≤ 10 мс на кадр для ROI layer	Sampling, smoothing, strict area cap.

11.8 Научный аспект: текстурированные ROI как приближение к perceptual saliency

Textured ROI не является полной saliency model. Тем не менее он приближает один важный аспект зрительного восприятия: чувствительность к потере мелкой структуры. Классические saliency-aware методы пытаются предсказать, куда смотрит пользователь [32]. Textured ROI решает более узкую и инженерно управляемую задачу: найти области, где высокочастотная структура является частью смыслового объекта. Одежда ведущего, узор, ворс и ткань не всегда являются самым важным объектом, но их деградация часто воспринимается как потеря резкости всей картинки.

Такое сужение задачи делает систему более практичной. Вместо общей saliency prediction, которая может быть нестабильной и трудно интерпретируемой, используется объектный детектор с понятной таксономией. Его ошибки легче анализировать через confusion matrix и per-class AP. Его outputs естественно переводятся в rectangular ROI. Его latency можно измерить и оптимизировать в C++.

11.9 Инженерный аспект: преимущество объединенной таксономии

Использование только DeepFashion2 дает большой объем данных, но объекты в нем чаще крупные и хорошо центрированные. Использование только Fashionpedia дает более разнообразную fashion ontology, но сильнее страдает от дисбаланса и меньшего объема. Объединенная таксономия дает компромисс: большой корпус, все 7 классов активны, редкие классы

получают больше примеров, а геометрия bbox становится более разнообразной.

С точки зрения обученной модели это важно: детектор текстурированных ROI должен работать не только на идеальных fashion-фотографиях, но и на видеокадрах, где одежда частично закрыта, движется, размывается или занимает нестандартную часть кадра. Поэтому merged dataset является более сильной практической базой, чем любой из датасетов по отдельности.

11.10 Ограничения валидности и требования к финальным экспериментам

Несмотря на высокие метрики YOLO26n, результаты обучения детектора текстурированных ROI нельзя автоматически считать доказательством улучшения видеостриминга. Детектор отвечает только за выделение областей, а итоговый пользовательский эффект возникает после кодирования. Поэтому финальная экспериментальная часть должна разделять две гипотезы. Первая гипотеза: модель достаточно надежно находит одежду и текстурированные области на кадрах, близких к streaming-контенту. Вторая гипотеза: передача этих областей кодировщику действительно повышает воспринимаемое качество при том же битрейте. Эти гипотезы связаны, но не эквивалентны.

Особенно важно контролировать domain shift. DeepFashion2 и Fashionpedia состоят из fashion-изображений, тогда как реальный видеостриминг включает motion blur, interlacing/resize artifacts, шум веб-камеры, неидеальное освещение, неполные фигуры, кадры с несколькими людьми и сильную компрессию до обработки. Поэтому в следующем этапе необходимо сформировать небольшой видеокорпус из реальных сценариев: talking-head, лекция с субтитрами, product review, live-stream fragment и видеоконференция. На этом корпусе следует проверить не только mAP, но и устойчивость ROI во времени.

Еще одно ограничение связано с тем, что модель обучена на прямоугольных bbox классов одежды. Для кодировщика прямоугольник является удобным форматом, но он может включать фон вокруг одежды. Если ROI слишком широкая, часть битов уйдет не на текстуру, а на окружающие области. Это решается двумя способами: более строгим clipping после NMS и ограничением площади ROI. В перспективе можно рассмотреть mask-based textured ROI, но для CPU-first версии прямоугольные ROI остаются более простым и дешевым решением.

Наконец, нужно учитывать, что высокий recall может быть важнее высокой precision. Если детектор текстурированных ROI ошибочно выделит небольшую лишнюю область, система потеряет часть битового бюджета. Если же он пропустит важную область одежды, пользователь увидит потерю детализации. Поэтому threshold в производственном режиме может отличаться от F1-optimal threshold. Для real-time кодирования допустимо работать в точке, где recall выше, но площадь ROI ограничена отдельным area cap. Такой дизайн отражает практическую природу работы: модель не классифицирует моду, а помогает кодировщику принять более разумное решение о распределении качества.

Заключение

В работе рассмотрена задача оптимизации воспринимаемого качества облачного видеостриминга в условиях ограниченных вычислительных ресурсов. Предложенный подход основан на ROI-aware обработке: система выделяет текст, лица и текстурированные области, нормализует результаты разных детекторов в геометрический формат *VecQuad*, а downstream-слой кодировщика должен дополнять эти области типом, приоритетом и qoffset-политикой.

Центральным результатом является архитектура библиотеки IDet. Библиотека рассматривается как CPU-first слой между компьютерным зрением и видеокодированием: она описывает жизненный цикл кадра, границы ответственности адаптеров, модель владения буферами, политику потоков, слияние ROI и преобразование приоритетов в *encoder guidance*. Такой подход важен для облачного сценария, где качество модели должно оцениваться вместе с задержкой, предсказуемостью памяти и возможностью интеграции в C++ pipeline.

Для третьего режима IDet подготовлен детектор текстурированных ROI на базе YOLO26n и воспроизводимый generic YOLO pipeline. Конвейер переводит DeepFashion2 и Fashionpedia в YOLO-формат, приводит их к общей 7-классовой таксономии, выполняет аудит данных, формирует отчеты, а также задает контракт экспорта PT-модели в ONNX-артефакт для последующего исполнения через ONNX Runtime. Экспериментальная часть подтверждает качество детекции выбранной легкой модели на объединенной 7-классовой таксономии и показывает CPU-only рабочие точки IDet: для textured ROI получена speed-first конфигурация с $p99=9.230$ мс и $\text{AreaF}_1 = 0.871$ на benchmark-изображении.

Научная составляющая работы состоит в формализации ROI-aware видеокодирования как ограниченной оптимизационной задачи, введении ROI-weighted качества, анализе area budget, temporal smoothing, qoffset mapping и метрик детекции в контексте видеокодирования. Практическая составляющая состоит в проектировании библиотеки IDet, подготовке textured ROI detector, описании deployment-контракта и формировании протокола оценки качества и задержки.

Итоговая архитектура не заменяет ABR-логику и не является самостоятельным кодировщиком. Она дополняет существующий видеоконвейер пространственным уровнем принятия решений: какие области кадра необходимо защищать в первую очередь и как передать эту информацию кодировщику. Именно это делает работу не только ML-экспериментом, но и инженерной ВКР о проектировании воспроизводимого ROI extraction layer для ресурсно-ограниченного облачного видеостриминга.

Основное ограничение текущих результатов состоит в том, что финальный вывод о качестве видеостриминга должен быть подтвержден отдельным экспериментом baseline против ROI-aware encoded stream. До получения этих чисел утверждение о росте качества следует формулировать как проверяемую инженерную гипотезу: ожидается, что ROI-aware поток улучшит ROI-weighted VMAF/SSIM и визуальное качество текста, лиц и фактур при контролируемом битрейте и допустимом p99 latency.

Список литературы

- [1] Spiteri K., Urgaonkar R., Sitaraman R. K. BOLA: Near-Optimal Bitrate Adaptation for Online Videos. IEEE INFOCOM, 2016.
- [2] Mao H., Netravali R., Alizadeh M. Neural Adaptive Video Streaming with Pensieve. ACM SIGCOMM, 2017.
- [3] Wang Z., Bovik A. C., Sheikh H. R., Simoncelli E. P. Image Quality Assessment: From Error Visibility to Structural Similarity. IEEE Transactions on Image Processing, 2004.
- [4] Netflix. VMAF: Video Multi-Method Assessment Fusion. URL: <https://github.com/Netflix/vmaf>.
- [5] Zhang R., Isola P., Efros A. A., Shechtman E., Wang O. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. CVPR, 2018.
- [6] Wiegand T., Sullivan G. J., Bjøntegaard G., Luthra A. Overview of the H.264/AVC Video Coding Standard. IEEE Transactions on Circuits and Systems for Video Technology, 2003.
- [7] ITU-T. Recommendation H.265: High Efficiency Video Coding. URL: <https://www.itu.int/rec/T-REC-H.265>.
- [8] Ватолин Д. С. Сжатие видео. H.264. Лекции лаборатории компьютерной графики и мультимедиа МГУ, 2005.
- [9] Университет ИТМО. Методы сжатия изображений, аудиосигналов и видео: учебное пособие, 2009.
- [10] FFmpeg Developers. AVRegionOfInterest Struct Reference. URL: <https://ffmpeg.org/doxygen/trunk/structAVRegionOfInterest.html>.
- [11] FFmpeg Filters Documentation. addroi video filter. URL: <https://ayosec.github.io/ffmpeg-filters-docs/6.0/Filters/Video/addroi.html>.
- [12] x265 Project. Command Line Options. URL: <https://x265.readthedocs.io/en/stable/cli.html>.
- [13] Zhou X., Li Z., Zhang Y., Wang S. Region-of-Interest Based Coding Scheme for Live Videos. Applied Sciences, 2024. URL: <https://www.mdpi.com/2076-3417/14/9/3823>.
- [14] Huang C.-W., Chen H.-Y., Chung K.-L. Region-of-interest determination and bit-rate conversion for H.264 video transcoding. EURASIP Journal on Advances in Signal Processing, 2013. URL: <https://link.springer.com/article/10.1186/1687-6180-2013-112>.

- [15] Meddeb M., Cagnazzo M., Pesquet-Popescu B. Region-of-interest-based rate control scheme for high-efficiency video coding. APSIPA Transactions on Signal and Information Processing, 2014. URL: <https://www.cambridge.org/core/journals/apsipa-transactions-on-signal-and-information-processing/article/region-of-interest-based-rate-control-scheme-for-high-efficiency-video-coding/A969A31DDA4FE604B92C0449A4213392>.
- [16] Liao M., Wan Z., Yao C., Chen K., Bai X. Real-time Scene Text Detection with Differentiable Binarization. AAAI, 2020. URL: <https://arxiv.org/abs/1911.08947>.
- [17] Liao M., Zou Z., Wan Z., Yao C., Bai X. Real-Time Scene Text Detection with Differentiable Binarization and Adaptive Scale Fusion. IEEE TPAMI, 2022. URL: <https://arxiv.org/abs/2202.10304>.
- [18] Guo J., Deng J., Lattas A., Zafeiriou S. Sample and Computation Redistribution for Efficient Face Detection. ICLR, 2022. URL: <https://arxiv.org/abs/2105.04714>.
- [19] Bai J. et al. ONNX: Open Neural Network Exchange. URL: <https://onnx.ai/>.
- [20] ONNX Runtime Documentation. URL: <https://onnxruntime.ai/docs/>.
- [21] ONNX Runtime. I/O Binding for Performance. URL: <https://onnxruntime.ai/docs/performance/tune-performance/iobinding.html>.
- [22] ONNX Runtime. Graph Optimizations. URL: <https://onnxruntime.ai/docs/performance/model-optimizations/graph-optimizations.html>.
- [23] ONNX Runtime. Tune performance. URL: <https://onnxruntime.ai/docs/performance/tune-performance/>.
- [24] ONNX Runtime. Thread management. URL: <https://onnxruntime.ai/docs/performance/tune-performance/threading.html>.
- [25] ONNX Runtime. Quantize ONNX models. URL: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>.
- [26] Ultralytics. Model Export with Ultralytics YOLO. URL: <https://docs.ultralytics.com/modes/export/>.
- [27] Ultralytics. ONNX Export for YOLO Models. URL: <https://docs.ultralytics.com/integrations/onnx/>.
- [28] Ultralytics. YOLO Performance Metrics. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics/>.
- [29] Ultralytics. Data Augmentation using Ultralytics YOLO. URL: <https://docs.ultralytics.com/guides/yolo-data-augmentation/>.

- [30] Ge Y., Zhang R., Wu L., Wang X., Tang X., Luo P. DeepFashion2: A Versatile Benchmark for Detection, Pose Estimation, Segmentation and Re-Identification of Clothing Images. CVPR, 2019.
- [31] Jia M., Shi M., Sirotenko M., Cui Y., Cardie C., Hariharan B., Adam H., Belongie S. Fashionpedia: Ontology, Segmentation, and an Attribute Localization Dataset. ECCV, 2020.
- [32] Hadizadeh H., Bajić I. V. Saliency-Aware Video Compression. IEEE Transactions on Image Processing, 2014.
- [33] NVIDIA. GeForce RTX 4090 specifications. URL: <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/>.
- [34] Red Hat. Automatic NUMA Balancing. Red Hat Enterprise Linux 7 Virtualization Tuning and Optimization Guide. URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/virtualization_tuning_and_optimization_guide/sect-virtualization_tuning_optimization_guide-numa-auto_numa_balancing.
- [35] Linux Kernel Wiki. perf: Linux profiling with performance counters. URL: <https://perf.wiki.kernel.org/>.
- [36] Arm Developer. Statistical Profiling Extension. URL: <https://developer.arm.com/documentation/102477/latest/>.
- [37] Yasin A. A Top-Down Method for Performance Analysis and Counters Architecture. IEEE ISPASS, 2014.
- [38] Arm Developer. Learn the architecture: Top-down methodology. URL: <https://developer.arm.com/documentation/102198/latest/>.
- [39] Parshin D. IDet: CPU-only C++ library for text/face/clothing detection and ROI pipelines. URL: <https://github.com/De-Par/IDet>.
- [40] Parshin D. IDet model zoo: text, face and clothing detector artifacts. URL: <https://github.com/De-Par/IDet/blob/main/docs/model-zoo.md>.
- [41] Parshin D. Generic YOLO training pipeline. URL: <https://github.com/De-Par/yolo-training-pipeline>.

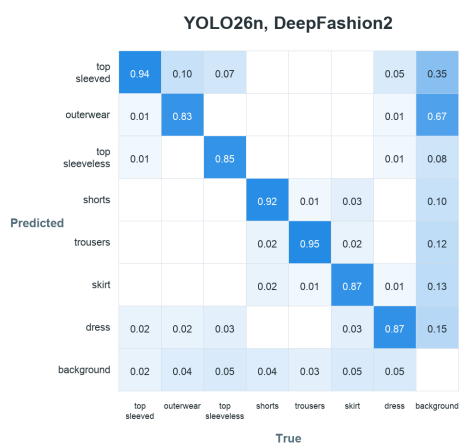
А. Дополнительные таблицы экспериментов

А.1 Сравнение датасетов

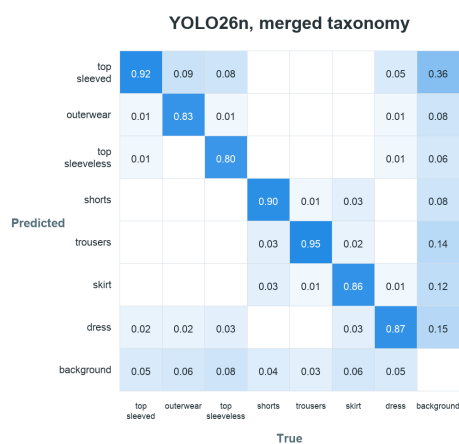
Таблица А.1: Расширенная сводка dataset audit

Датасет	Images	Instances	Inst/img	Classes	Empty	Invalid
DeepFashion2	224114	364675	1.6272	7/7	0	0
Fashionpedia	45445	74724	1.6443	7/7	0	0
Объединенный набор	269559	439399	1.6301	7/7	0	0

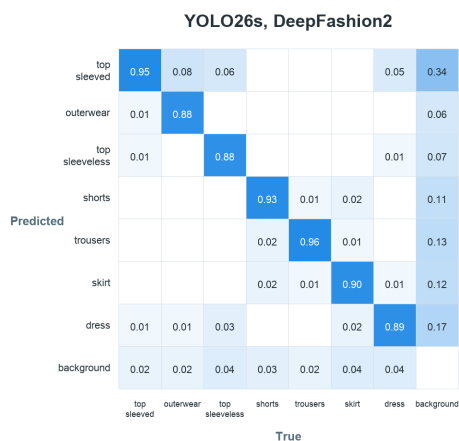
А.2 Матрицы ошибок YOLO-экспериментов



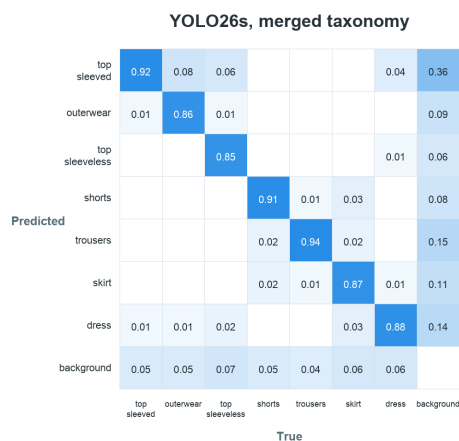
(a) YOLO26n, DeepFashion2



(b) YOLO26n, объединенный набор



(c) YOLO26s, DeepFashion2



(d) YOLO26s, объединенный набор

Рисунок А.1: Нормализованные матрицы ошибок для четырех YOLO-экспериментов.

A.3 Параметризованный запуск C++ benchmark

```
1 idet_app \  
2   --mode cloth \  
3   --model <onnx-model> \  
4   --image <input-image> \  
5   --bind_io 1 \  
6   --runtime_policy 1 \  
7   --soft_mem_bind 1 \  
8   --cpu_placement latency \  
9   --bench_iters 30 \  
10  --warmup_iters 5 \  
11  --is_draw 0 \  
12  --is_dump 0 \  
13  --verbose 0 \  
14  --fixed_hw 128x128 \  
15  --tiles_rc 2x4 \  
16  --threads_intra 8 \  
17  --threads_inter 1 \  
18  --tile_omp 8
```

Листинг A.1: Параметризованный запуск idet_app для чистого latency benchmark

A.4 Целевая процедура benchmark

```
1 1. Build a full-image reference configuration.  
2 2. For each candidate configuration:  
3   a. run idet_app with --is_draw 0 --is_dump 0 --verbose 0;  
4   b. parse p50_ms, p90_ms, p95_ms, p99_ms from stdout;  
5   c. reject candidates above the latency threshold;  
6   d. run a separate dump pass with --is_dump 1;  
7   e. extract Quads from stdout;  
8   f. compute AreaRecall, AreaPrecision, AreaF1,  
9       ExtraAreaRatio against the reference boxes.  
10 3. Sort accepted candidates by p99_ms and inspect the  
11    Pareto trade-off between p99 latency and area quality.
```

Листинг A.2: Алгоритмический benchmark-протокол grid search